

VNUHCM - UNIVERSITY OF SCIENCE
Faculty of Information Technology



Design and Implementation of the Hybrid FTP Application

CSC10008 Computer Networking – Summer 2026

Instructors: Ms. Chung Thùy Linh

Group members: Đỗ Minh Khang, Lâm Hồng Hải Hoàng Lê, Mai Phúc Hưng

Contents

1.	Application Scenario & Protocol Interaction.....	3
1.1.	Application Scenario.....	3
1.2.	Full Session Lifecycle.....	4
1.3.	Unique/Append Uploads and Transfer Cancellation	9
2.	Project-Wide Data Structures.....	10
2.1.	TCP Control Packet Format.....	10
2.2.	UDP Custom Header – RdtHeader	11
2.3.	Session Management Structures	14
2.4.	Filesystem & Application-Level Structures.....	17
2.5.	RDT Sender-Internal Tracking Structure.....	18
3.	Functional Workflows.....	19
3.1.	Server Thread-Dispatch Logic.....	19
3.2.	Reliable UDP Sender/Receiver State Machines	22
3.3.	Active/Passive Mode Toggle	28
3.4.	Unified Upload Handling: STOR / STOU / APPE.....	30
4.	Task Assignment Matrix	32
4.1.	Ownership Overview	32
4.2.	Layer Summary.....	34
5.	Self-Assessment & Peer Evaluation	35
5.1.	Lâm Hồng Hải Hoàng Lê.....	35
5.2.	Mai Phúc Hưng	36
5.3.	Đỗ Minh Khang	37
5.4.	Contribute Distribution	38
6.	GenAI Usage & Code Refinement Log.....	39
7.	Application Demo Evidence	39
8.	Tools used	42
8.1.	Libraries	42
8.2.	Applications/Websites.....	43

List of Figures:

Figure 1: PASV Process	6
Figure 2: PORT Process.....	8
Figure 3. ABOR Process	10
Figure 4: Server Thread-Dispatch.....	20
Figure 5: Transfer Worker Launch & Cross-Thread Handoff.....	21
Figure 6: sendChunk High-Level Flow	23
Figure 7: pollAcksAndRetransmit Subroutine.....	24
Figure 8: receiveNext Flow	25
Figure 9: Deferred Final-ACK on Download (RETR, Client as Receiver).....	27
Figure 10: Active/Passive Mode Toggle	29
Figure 11: UploadKind Destination Resolution.....	30
Figure 12: APPE Post-Receive Combination (appendAtomically)	31

1. Application Scenario & Protocol Interaction

1.1. Application Scenario

This project implements a **Hybrid FTP (File Transfer Protocol)** application that separates the control plane from the data plane, mirroring the architectural philosophy of RFC 959 while replacing the data channel's transport with a custom, application-layer reliability protocol built on raw UDP sockets.

The system consists of two independently-running processes:

- **ftp_server.exe** – a multi-threaded TCP server, listening on **port 4567**, spawning one dedicated thread per connected client to fully isolate session state (current directory, authentication status, active transfer mode) between concurrent users.
- **ftp_client.exe** – an interactive command-line client that connects to the server's control port, issues FTP-style commands, and drives file transfers over a separately-established UDP data channel.

Two logically distinct channels exist between any connected client and the server for the duration of a session:

Channel	Transport	Purpose	Established by
Control	TCP	Command/reply exchange (login, navigation, transfer negotiation)	OS-level 3-way handshake on client connect
Data	UDP (w/custom RDT)	Actual file payload for RETR/STOR	Negotiated per-transfer via PASV/PORT

The control channel remains open for the entire session (until QUIT); the data channel is opened fresh for each individual file transfer and closed once that transfer completes.

1.2. Full Session Lifecycle

The following phases are common to every session, regardless of which data-channel mode is eventually used, and precede the PASV/PORT split covered in 1.2.1 and 1.2.2.

Phase 1 – Control Connection Establishment: The client initiates a standard TCP three-way handshake to the server's port 4567, handled entirely by the OS socket stack.

Phase 2 – Greeting & Authentication: Immediately upon accepting the connection, the server sends an unsolicited “220 Service ready” reply. The client sends `USER <name>`; the server checks the username against a credential store and replies “331” (username acknowledged, password required) only if it exists. The client then sends `PASS <password>`; the server compares it against the stored password for that username, replying “230 User logged in” on a match or “530 Not logged in, password incorrect” otherwise. Every command other than `USER/PASS/HELP/NOOP/QUIT` is gated behind successful authentication.

Phase 3 – Command Phase (Navigation): The client may issue any number of control-only commands – `PWD`, `CWD`, `LIST`, `MKD`, `SIZE`, `STAT`, etc. – each producing exactly one TCP reply line, with no data-channel involvement.

At this point the session branches depending on which data-channel mode the client selects for its next transfer.

1.2.1. Passive Mode (PASV) – Data Channel Negotiation & Transfer

Phase 4 – Data Channel Negotiation: The client sends PASV. The server opens a new UDP socket bound to an OS-assigned ephemeral port and replies “227 Entering Passive Mode (h1,h2,h3,h4,p1,p2)”, encoding its own IP and the chosen port. This socket is kept open and reused directly by the RDT transport.

Phase 5 – Transfer Initiation and Handshake (direction-dependent): The client sends RETR <filename> or STOR <filename>; the server replies “150 Opening data connection”.

- For **RETR** (server is the sender): the server's socket is only bound, not connected to a specific peer, so it cannot send file data until it knows the client's address. It blocks waiting for an initial FLAG_SYN handshake packet, which the client sends immediately after connecting to the server's advertised address.
- For **STOR** (server is the receiver): no handshake is required – the client already knows the server's address (it connected to it), so it can begin sending FLAG_DATA packets immediately; the server's receiver learns the client's address naturally from the first packet it receives via `recvfrom()`.

Phase 6 – Reliable Data Transfer: The file is read in fixed-size chunks and sent as a sequence of UDP packets, each carrying a sequence number, flags, and a 16-bit checksum. The sender maintains a sliding window (Selective Repeat), retransmitting any packet whose acknowledgment is not received within a dynamically-estimated timeout (derived from measured RTT). The receiver acknowledges every valid packet it receives – including duplicates – and silently discards any packet that fails its checksum. The final chunk is marked with an explicit FLAG_FIN flag so the receiver knows to stop and finalize the file.

Phase 7 – Integrity Verification: Once the transfer completes, the sending side computes a SHA-256 digest of the file just sent/received and embeds it directly in the completion reply “226 Transfer complete. SHA256=<64-hex-digest>”. The receiving side independently computes the hash of its own resulting file and compares the two values, reporting a match or mismatch locally – this step applies to Binary-mode transfers only.

Phase 8 – Session Termination: The client sends QUIT; the server replies “221 Service closing control connection” and closes the socket.

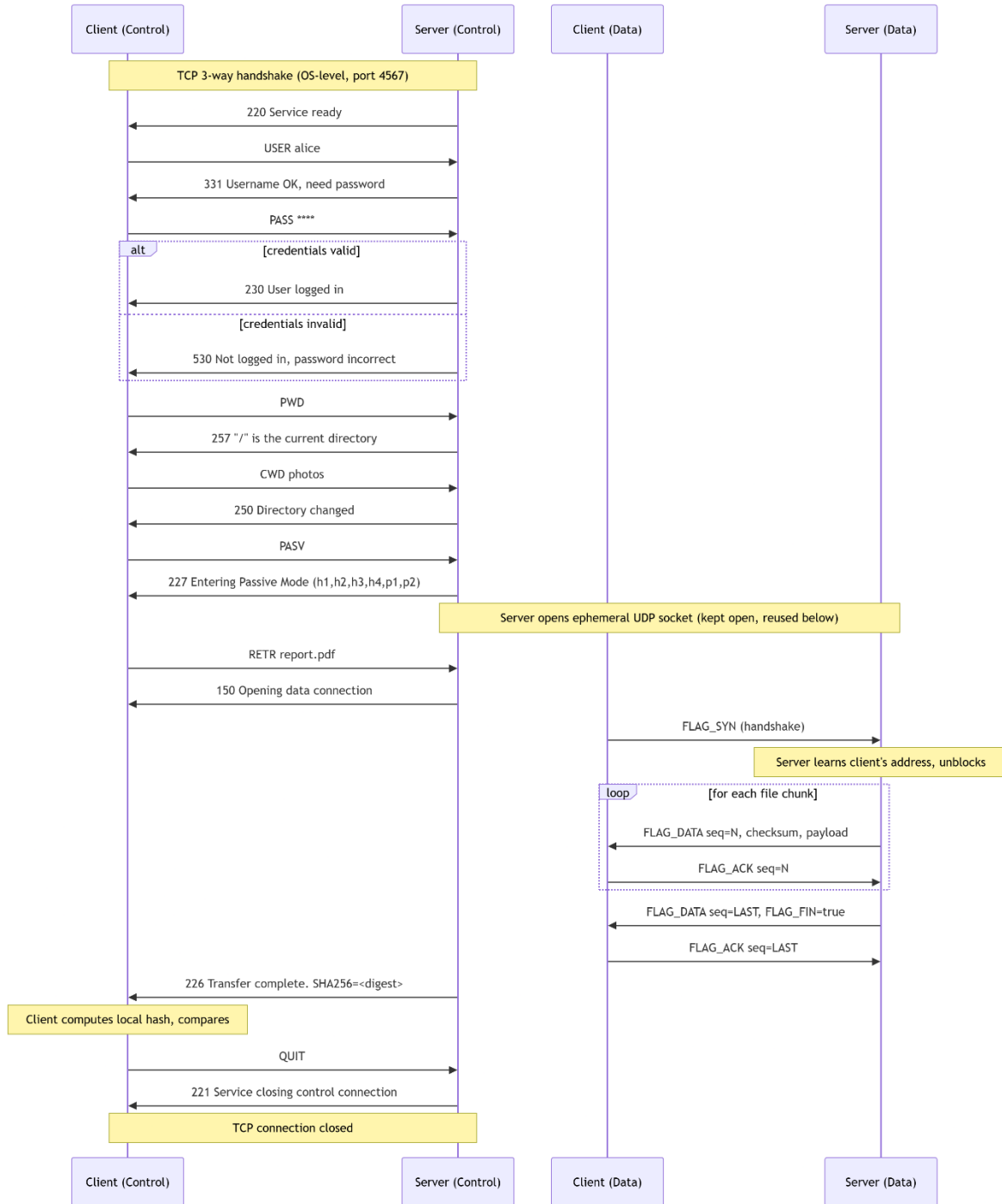


Figure 1: PASV Process

1.2.2. Active Mode (PORT) – Data Channel Negotiation & Transfer

Phase 4 – Data Channel Negotiation: The client opens and binds its own UDP socket on an OS-assigned ephemeral port, determines its own local IP address (by probing a throwaway socket connected toward the server), and sends “PORT h1,h2,h3,h4,p1,p2” advertising that address. The server replies “200 PORT command successful”.

Phase 5 – Transfer Initiation and Handshake (direction-dependent):

- For **RETR** (server is the sender): the server already knows the client's exact address from the parsed PORT command, so it can begin sending FLAG_DATA packets immediately – no handshake is required.
- For **STOR** (server is the receiver): the server does not automatically have a rendezvous point to receive on, so it performs a **server-initiated symmetric handshake** – it sends a FLAG_SYN packet to the client's PORT-advertised address before expecting any data. The client, already bound and listening on that address, receives the SYN and begins sending file data in response.

Phase 6 – 8: Proceed identically to the Passive-mode case (reliable windowed transfer, hash-verified completion, session termination).

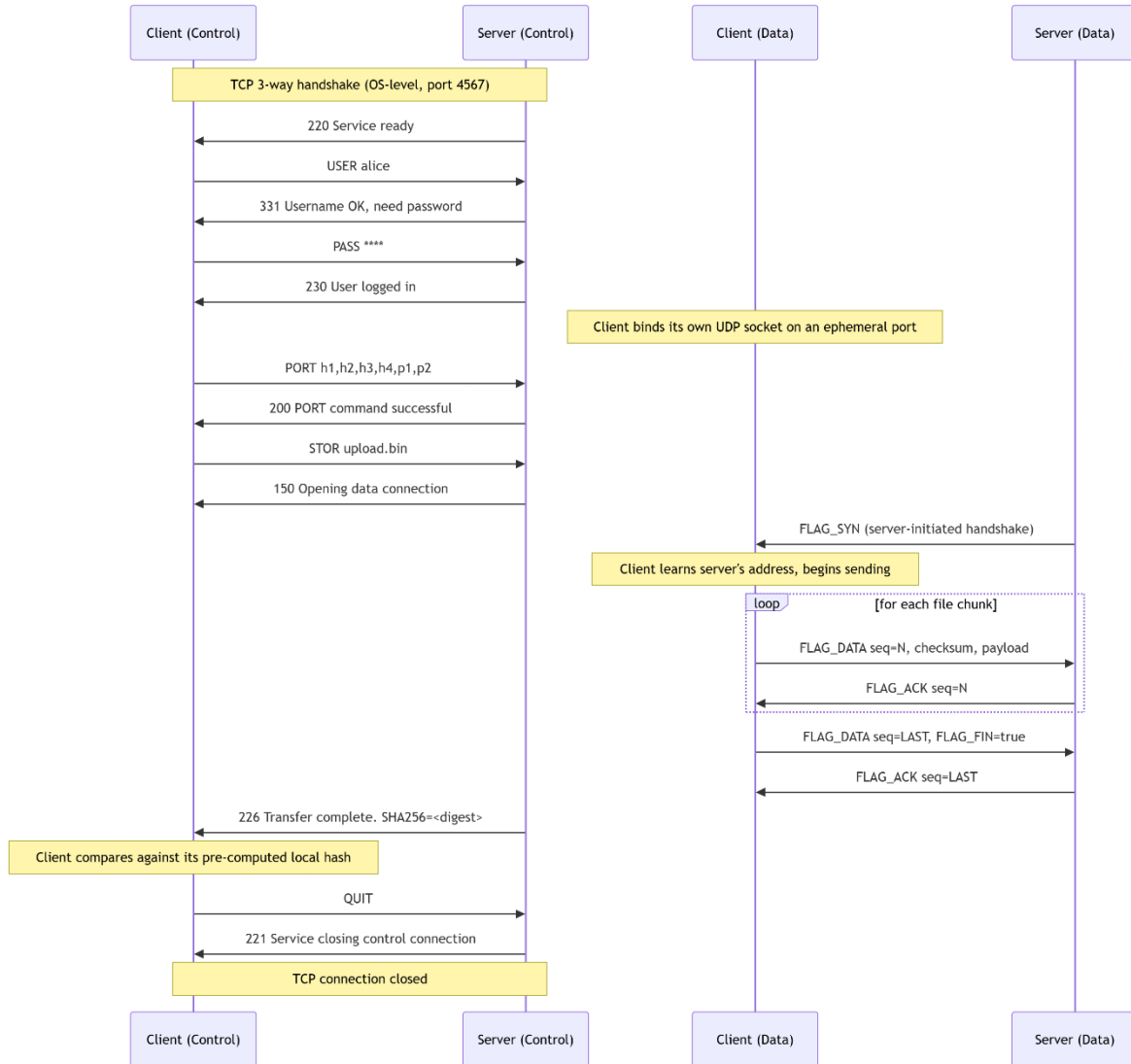


Figure 2: PORT Process

1.3. Unique/Append Uploads and Transfer Cancellation

Three further commands extend the transfer lifecycle described in 1.2.1/1.2.2, and one changes its execution model:

MODE {S|B|C} is accepted for Stream mode (S) – the mode this project already implements throughout – and explicitly rejected with “504 Only MODE S is supported.” for that parameter for Block (B) or Compressed (C), rather than silently ignoring or mishandling an unsupported request.

STOU <filename> behaves exactly like STOR from the client's perspective, except the server generates and reserves a guaranteed-unused filename rather than trusting a client-supplied one; that filename is reported back to the client in the 226 reply, since the client has no other way to learn what name was actually used.

APPE <filename> behaves like STOR up through the point where the new data has been fully and correctly received into a staging area; only once that succeeds does the server combine the newly-received data with any existing file at the destination (or simply use it as-is if the destination doesn't yet exist), committing the combined result atomically. A failed or aborted APPE leaves any pre-existing destination file completely untouched.

ABOR stops any data transfer process (RETR/STOR/STOU/APPE) by sets a shared, thread-safe flag and then blocks until the worker thread confirms it has actually stopped before replying, giving the client a verified cancellation. More information about the worker thread is provided in section [3.1](#).

While a transfer worker is active, the control loop doesn't simply ignore other commands – `executeCommand` explicitly rejects any command other than ABOR with “503 Transfer already in progress; use ABOR first”. (see Figure 4), so a client can't accidentally issue a second RETR on top of one still running.

The worker thread's completion routine (`finishTransfer`) checks whether the abort was client-requested before it replies. If `abortRequested` is **not set** – i.e. the transfer simply finished or failed on its own – the worker sends the 226/426 reply itself. But if `abortRequested` is set, the worker sends nothing at all; it only flips `transferActive` back to false and exits. The “426 Transfer aborted”. reply is instead sent by the ABOR command handler itself, on the control thread, immediately after `join()` returns. This avoids a race where both threads could try to write the final reply, and it guarantees the client always sees the literal text “Transfer aborted.” for a user-initiated cancellation rather than whatever generic failure text the worker would otherwise have produced.

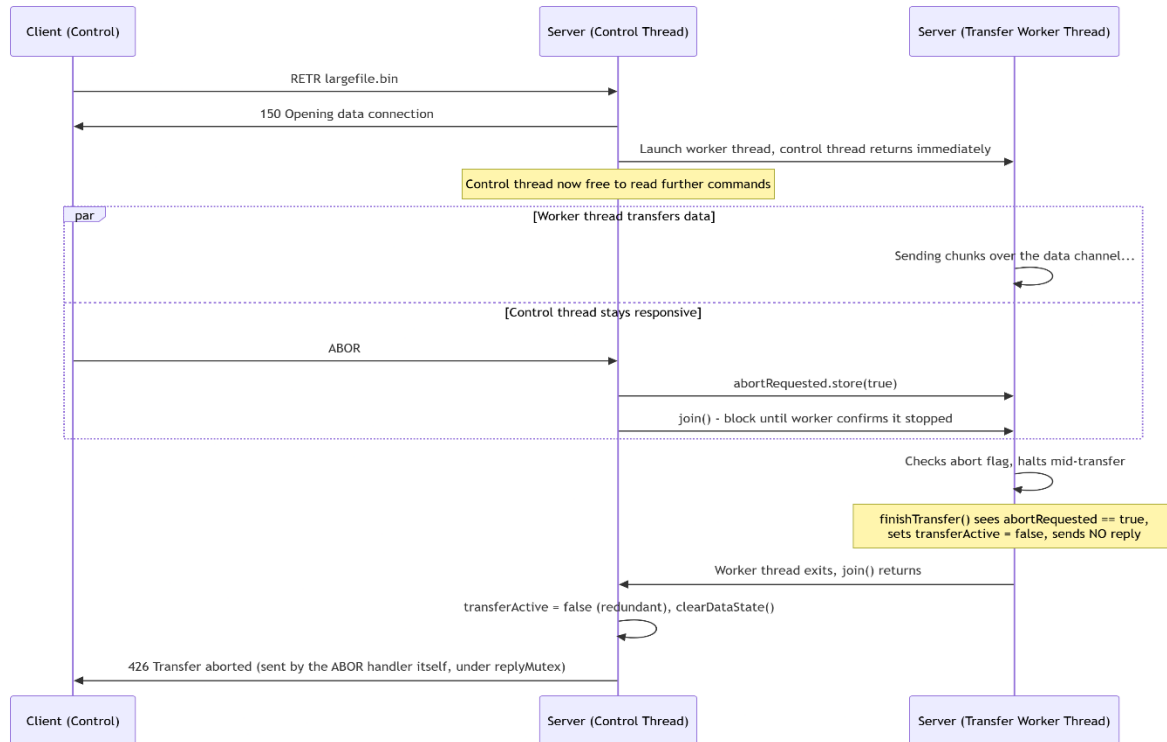


Figure 3. ABOR Process

2. Project-Wide Data Structures

2.1. TCP Control Packet Format

Unlike the UDP data channel, the control channel does not use a fixed binary structure. Commands and replies are plain, `\r\n`-terminated text lines, consistent with the human-readable convention used by real-world FTP.

2.1.1. Client → Server (Command) Format

`<VERB> [argument]\r\n`

Parsed server-side by `CommandParser::parseCommand`, which splits the line into a `ParsedCommand` structure:

Field	Type	Purpose
<code>rawCommand</code>	<code>std::string</code>	The full, untouched input line
<code>type</code>	<code>std::string</code>	The command verb, uppercased (e.g. "RETR")
<code>args</code>	<code>std::string</code>	Whitespace-separated arguments following the verb

2.1.2. Server → Client (Reply) Format

```
<code> <message>\r\n
```

Produced by `Session::replyWithCode`, which constructs the line directly as:

```
std::to_string(replyCode) + " " + message + "\r\n"
```

Produced by `Session::replyWithCode` as `std::to_string(replyCode) + " " + message + "\r\n"`. A multi-line variant, `Session::multilineReplyWithCode` (used for LIST/NLST output), splits a message containing embedded `\n` characters into multiple physical lines, each re-terminated with `\r\n` – this is a deliberately simplified, project-internal convention rather than RFC 959's hyphenated continuation format, meaning replies from this server are not intended to be parsed by a strict RFC-959-compliant client.

2.2. UDP Custom Header – RdtHeader

The fixed, 16-byte binary structure prefixing every UDP datagram on the data channel, defined once in `shared/protocol/RdtHeader.h` and used identically by both client and server.

2.2.1. Byte-Level Layout

Offset (bytes)	Field	Type	Size (bytes)	Wire byte order
0–3	seq_num	uint32_t	4	Network (big-endian) – htonl / ntohl
4–7	ack_num	uint32_t	4	Network (big-endian) – htonl / ntohl
8	flags	uint8_t	1	N/A (single byte)
9–10	window_size	uint16_t	2	Network (big-endian) – htonl / ntohl
11–12	payload_len	uint16_t	2	Network (big-endian) – htonl / ntohl
13–14	checksum	uint16_t	2	Network (big-endian) – htonl / ntohl
15	reserved	uint8_t	1	N/A (padding/future use)

Total header size: 16 bytes, enforced at compile time via `static_assert(HEADER_SIZE == 16, ...)` and `#pragma pack(push, 1)` (disabling compiler struct-padding, guaranteeing the in-memory layout matches this table exactly with no hidden gaps).

Multi-byte fields are explicitly converted to network byte order field-by-field during serialization (`serializeHeader`) and converted back during `deserializeHeader` – this is a manual, explicit byte-order conversion rather than relying on the struct's raw in-memory representation, which is necessary because host byte order is not guaranteed to match network byte order (x86/x64 is little-endian; the wire format must be big-endian regardless of host).

2.2.2. flags Bit Layout

Bit	Flag	Value	Meaning
0	FLAG_SYN	0x01	Handshake-initiation packet (used for Passive-mode RETR and Active-mode STOR peer discovery)
1	FLAG_ACK	0x02	Acknowledgment packet
2	FLAG_FIN	0x04	Marks the final chunk of a transfer
3	FLAG_DATA	0x08	Carries file payload
4	FLAG_NAK	0x10	Negative acknowledgment (explicit reject signal)
5–7	–	–	Unused

2.2.3. RdtPacket – Header + Payload

```
struct RdtPacket {  
    RdtHeader header;           // 16 bytes  
    char payload[MAX_PAYLOAD]; // 1024 bytes  
};
```

Total maximum wire size per datagram: **1040 bytes** (16-byte header + up to 1024 bytes of payload), comfortably under typical Ethernet MTU (1500 bytes), avoiding IP fragmentation.

2.2.4. Related Protocol Constants

Reside at `shared/protocol/ProtocolConstants.h`

Constant	Value	Purpose
MAX_PAYLOAD	1024	Maximum file-data bytes per packet; single source of truth shared with the filesystem layer's chunk size
HANDSHAKE_TIMEOUT_MS	500	Timeout while waiting for a FLAG_SYN handshake reply
HANDSHAKE_MAX_RETRIES	10	Maximum handshake retry attempts before aborting
DATA_IDLE_TIMEOUT_MS	5000	Receiver's maximum idle wait for the next packet under normal conditions
CHAOS_DATA_IDLE_TIMEOUT_MS	30000	Extended idle wait used specifically when chaos-mode fault injection is active
RDТ_RECEIVE_WINDOW	10	The receiver's out-of-order lookahead window; a packet whose sequence number is $\geq \text{expected_seq} + \text{RDТ_RECEIVE_WINDOW}$ is dropped without an ACK (Section 3.2.2 , Figure 8)

(Congestion-control parameters are documented in Section [3.2.1](#), alongside the sender state machine they govern.)

2.2.5. IRdtTransport – Shared Transport Interface

Reside at `shared/protocol/RdtHeader.h`. `RdtSender` and `RdtReceiver`, on both sides of the connection, implement a common abstract interface rather than being used through their concrete types directly. This is what lets `DataChannelSession::sendFile/receiveFile` (Section 3.4) stay agnostic to which transport variant was constructed for a given transfer.

Method	Purpose
<code>sendChunk(seqNum, data, len, isFinal)</code>	Send one chunk; implemented meaningfully only by <code>RdtSender</code>
<code>receiveNext(outSeqNum, outData, outIsFinal)</code>	Receive the next chunk; implemented meaningfully only by <code>RdtReceiver</code>
<code>confirmReceive(seqNum, accepted)</code>	Lets a receiver delay its final ACK until the data has been durably committed locally, and reject a sequence number outright by returning <code>false</code> /sending a NAK. The base interface default is a no-op that echoes <code>accepted</code> back without sending anything on the wire – see Section 3.2.3 for the one implementation (the client's <code>RdtReceiver</code>) that actually overrides it
<code>flush()</code>	Drain any remaining in-flight state; meaningful only for <code>RdtSender</code>

2.3. Session Management Structures

2.3.1. Server – ClientSession

Reside at `server/src/control/Session.h`:

Field	Type	Purpose
<code>socket</code>	<code>SOCKET</code>	This client's control-channel socket
<code>authenticated</code>	<code>bool</code>	Login status, set only after a real credential check against <code>Users::database</code>
<code>transferMode</code>	<code>TransferMode</code>	Current TYPE A/TYPE I setting
<code>username</code>	<code>std::string</code>	Username supplied via USER
<code>currentDir</code>	<code>std::filesystem::path</code>	This session's current working directory, bounded by <code>PathResolver</code>

<code>dataChannelMode</code>	<code>DataChannelMode</code>	None / Passive / Active – which mode the next transfer will use
<code>pendingDataSocket</code>	<code>SOCKET</code>	The UDP socket opened by PASV, reused directly by the RDT transport
<code>pendingPeerAddr</code>	<code>sockaddr_in</code>	Client's advertised address (Active mode) or discovered peer address (Passive mode)
<code>controlPeerAddr</code>	<code>sockaddr_in</code>	The client's control-connection address, used to validate PORT requests
<code>pendingRenameSource</code>	<code>std::filesystem::path</code>	Resolved source path held between RNFR and RNT0
<code>hasPendingRename</code>	<code>bool</code>	Whether a RNFR is currently awaiting its matching RNT0
<code>transferActive</code>	<code>std::atomic<bool></code>	Whether a transfer worker thread is currently running for this session
<code>abortRequested</code>	<code>std::atomic<bool></code>	Set by the ABOR handler; polled by the transfer worker (and, via the same predicate, by the RDT layer itself) to signal early termination
<code>transferWorker</code>	<code>std::thread</code>	The detached-in-purpose (but explicitly joined) worker thread running the current RETR/STOR/STOU/APPE transfer, decoupling it from the control-command loop
<code>replyMutex</code>	<code>std::mutex</code>	Guards concurrent writes to the control socket, since the transfer worker now sends its own completion reply independently of the control thread
<code>pendingUniqueFilename</code>	<code>std::string</code>	The server-generated filename reserved by STOU; reported back to the client only in the completion (226) reply (Section 1.3)

2.3.2. Client – ClientState

Reside at `client/src/control/Session.h`:

Field	Type	Purpose
<code>dataMode</code>	<code>DataMode</code>	Client-side equivalent of <code>DataChannelMode</code>
<code>pendingDataSocket</code>	<code>SOCKET</code>	The client's own data-channel socket
<code>passiveAddress</code>	<code>sockaddr_in</code>	Server's address, parsed from a 227 reply
<code>serverAddress</code>	<code>sockaddr_in</code>	The control-connection server address
<code>transferMode</code>	<code>TransferMode</code>	Mirrors the server's TYPE setting locally
<code>replyBuffer</code>	<code>std::string</code>	Accumulates partial TCP reads until a full <code>\r\n</code> -terminated reply line is available

2.3.3. Transfer Coordination Types

Introduced alongside the asynchronous transfer-worker model and the unification of STOR/STOU/APPE into a single code path.

SendResult, reside at `server/src/datachannel/DataChannelSession.h`, is the return type of `sendFile`, replacing a bare `bool`:

Field	Type	Purpose
<code>success</code>	<code>bool</code>	Whether the transfer completed successfully (also exposed via operator <code>bool()</code> for direct use in conditionals)
<code>sha256</code>	<code>std::string</code>	The SHA-256 digest computed from the sender's source snapshot, empty if hashing failed or is not applicable

UploadKind, reside at `server/src/control/CommandDispatcher.cpp` in a anonymous namespace, is an enum distinguishing the three upload commands that now share one implementation (`startUploadTransfer`):

Value	Corresponding command	Behavior
Store	STOR	Writes to the client-supplied filename, overwriting if it exists
Unique	STOU	Writes to a server-reserved, guaranteed-unused filename
Append	APPE	Combines new data with any existing file at the destination, rather than overwriting it

2.4. Filesystem & Application-Level Structures

2.4.1. DirEntryInfo

Reside at `server/src/filesystem/DirectoryService.h`:

Field	Type	Purpose
<code>formatPermissions</code>	<code>std::string</code>	Unix-style permission string (e.g. <code>-rw-r--r--</code>) for LIST output
<code>name</code>	<code>std::string</code>	Entry filename
<code>isDirectory</code>	<code>bool</code>	Directory vs. regular file
<code>sizeBytes</code>	<code>uintmax_t</code>	File size (0 for directories)

2.4.2. DirectoryService::PathMetadata

Field	Type	Purpose
<code>exists</code>	<code>bool</code>	Whether the resolved path exists
<code>isDirectory</code>	<code>bool</code>	Directory vs. file
<code>sizeBytes</code>	<code>bool</code>	File size
<code>lastModified</code>	<code>uintmax_t</code>	Backing MDTM's timestamp reporting

2.4.3. DirectoryService::Result

Field	Type	Purpose
ok	bool	Success/failure
code	int	FTP reply code to send
message	std::string	Human-readable reply text

2.5. RDT Sender-Internal Tracking Structure

2.5.1. InFlightPacket

Reside (private) at server/src/rdt/RdtSender.h:

Field	Type	Purpose
seq_num	uint32_t	This packet's sequence number
data	std::vector<char>	Buffered payload, retained for possible retransmission
is_final	bool	Whether this is the transfer's last chunk
sent_time	std::chrono::steady_clock::time_point	Used for timeout detection and RTT sampling
acked	bool	Whether this specific packet has been acknowledged
retransmitted	bool	Whether this packet has been resent – excludes it from RTT sampling (Karn's Algorithm)
retries	int	Count toward MAX_RETRIES

Held in `std::deque<InFlightPacket>` window – the sender's Selective Repeat sliding window; its slide condition and congestion-window interaction are detailed in Section [3.2.1](#).

3. Functional Workflows

3.1. Server Thread-Dispatch Logic

The server's concurrency model is thread-per-connection: `runSession()` binds and listens on port 4567, then loops on `accept()`, spawning a detached `std::thread` running `handleClient()` for every accepted connection and immediately returning to `accept()` for the next one.

Inside `handleClient()`, the thread constructs a fresh `ClientSession`, registers the connection in a shared `ClientRegistry`, and logs the connection via `Logger`. A `RegistryGuard` RAI object guarantees the client is removed from the registry and the disconnect is logged on every exit path, including early returns.

The thread then loops: read from the socket into a buffer, extract one complete newline-terminated command at a time, parse it, and dispatch it. After every dispatched command, the loop checks whether `session.socket` was invalidated (set by the QUIT handler) and exits the thread cleanly if so, closing any still-open data-channel socket first.

For most commands, dispatch is still synchronous – the handler runs to completion and the loop immediately reads the next command. For RETR/STOR/STOU/APPE specifically, dispatch instead **launches a second, dedicated worker thread** to run the actual transfer and returns immediately, so the control loop stays free to receive further commands – ABOR – while a transfer is still in progress.

The claim that "the worker thread sends its own completion reply once finished, independently of the control loop" is only true when the transfer finishes or fails **on its own**. When the client sends ABOR, the worker sends no reply at all and the control thread's ABOR handler sends the final 426 itself after `join()` returns – see the walkthrough in Section [1.3](#) and Figure 3. `ClientSession::replyMutex` still guards every write to the control socket regardless of which thread is doing the writing, so the two threads can never corrupt each other's output.

`executeCommand` performs two checks before it will look a command up at all. First, an authentication gate (every command except USER/PASS/HELP/NOOP/QUIT requires `session.authenticated`). Second, a **busy guard**: if a transfer worker is currently active, every command *except* ABOR is rejected outright with "503 Transfer already in progress; use ABOR first.", without even reaching the command table. Only after both checks pass does the loop call `joinCompletedTransfer()` – an opportunistic, non-blocking reap of a worker thread that has already set `transferActive = false` but whose `std::thread` handle hasn't been joined yet, so the next transfer on this session doesn't accumulate stale joinable threads.

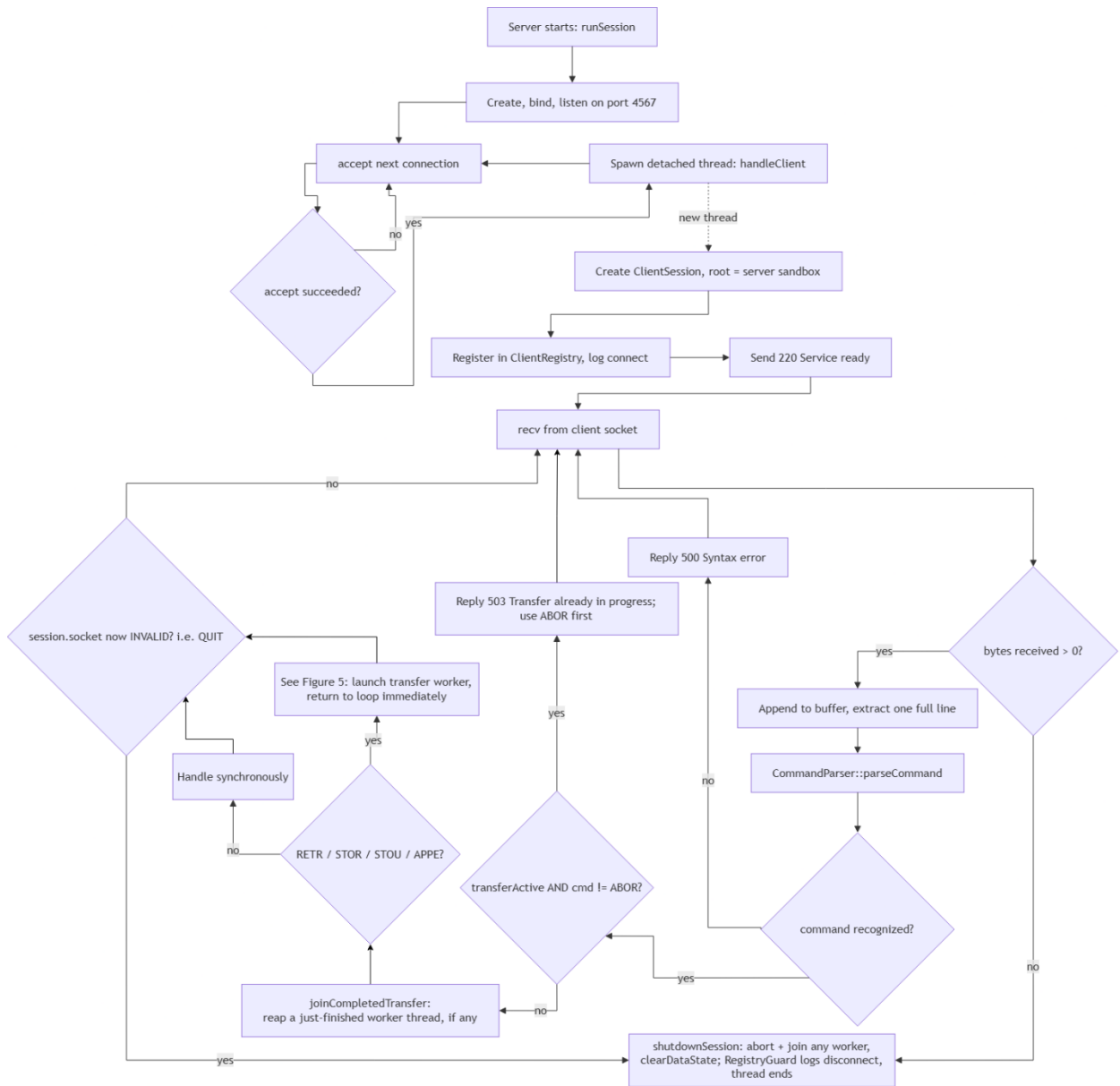


Figure 4: Server Thread-Dispatch

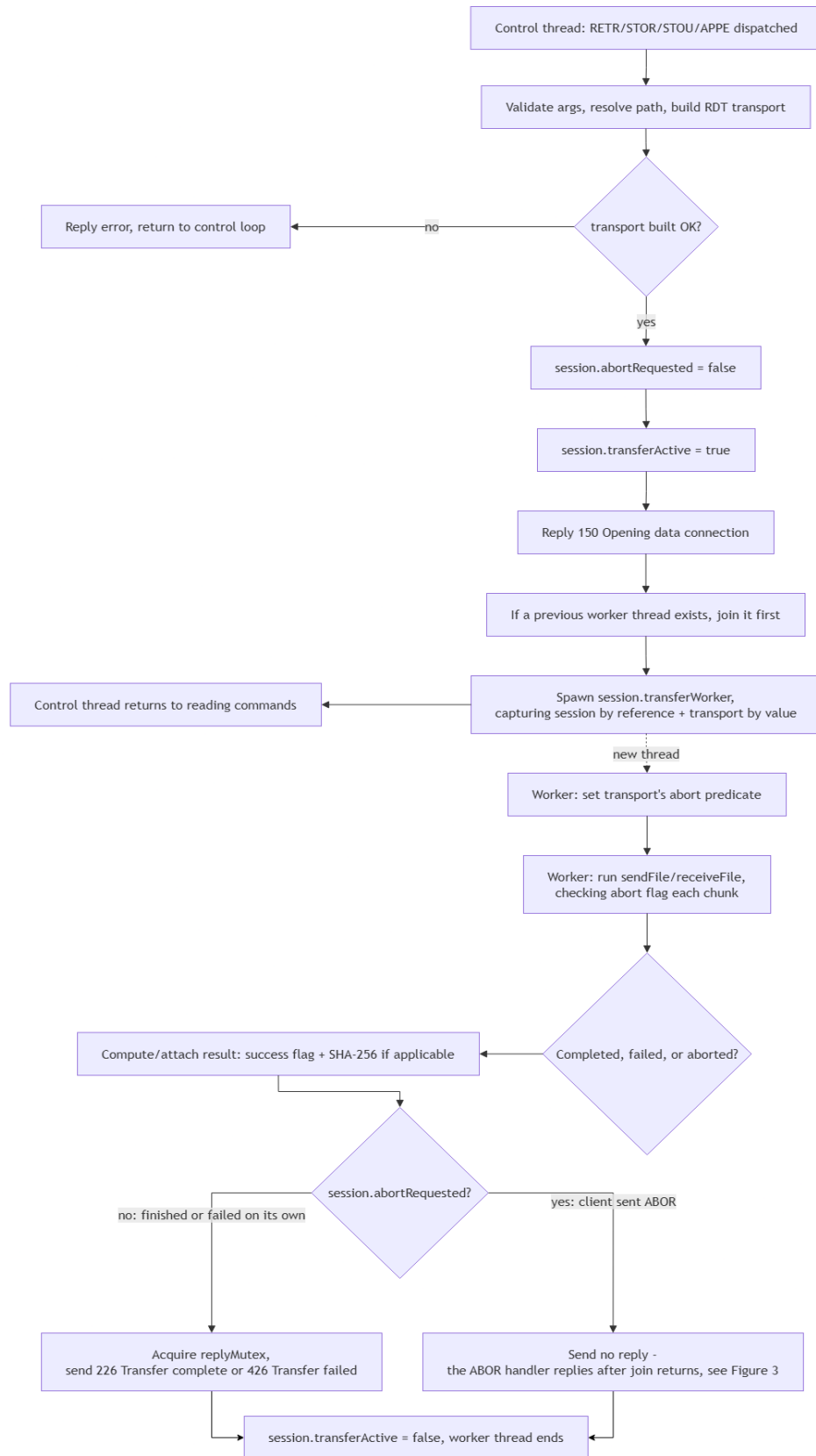


Figure 5: Transfer Worker Launch & Cross-Thread Handoff

3.2. Reliable UDP Sender/Receiver State Machines

Both `RdtSender` and `RdtReceiver` implement Selective Repeat over a sliding window, with `RdtSender` additionally performing RTT-based dynamic timeout estimation and AIMD (Additive-Increase/Multiplicative-Decrease) congestion control.

`RdtSender/RdtReceiver` exist as **separate, independently-maintained implementations** under `server/src/rdt/` and `client/src/rdt/` – not one shared class used by both sides. The two copies are functionally very close, but not identical. Section 3.2.3 documents one genuine behavioral asymmetry between them.

3.2.1. `RdtSender` – Selective Repeat Send + Congestion Control

`RdtSender` has two constructors: one that creates and connects a fresh UDP socket (Active-mode RETR), and one that reuses an already-bound socket handed in by the caller (Passive-mode RETR).

`sendChunk` first rejects any payload exceeding `MAX_PAYLOAD`, then blocks – via repeated calls to `pollAcksAndRetransmit(blocking=true)` – while the in-flight window is full. Once space is available, it builds the packet, transmits it, records it in the window, and performs one quick non-blocking poll for already-arrived ACKs before returning.

`pollAcksAndRetransmit` handles two responsibilities in one pass: draining available ACKs and updating window/RTT/congestion state, then scanning for any unacked packet whose timeout has elapsed and retransmitting it – or aborting the transfer entirely if that packet has exhausted `MAX_RETRIES`. Every incoming datagram is checked against the sender's recorded `destAddr` (via a `sameEndpoint` helper comparing family, port, and address) before it's treated as a real ACK/NAK – a datagram from any other source is simply ignored rather than processed, closing off a spoofing/cross-talk vector on a socket that, in Active mode, isn't `connect()`-ed to a single peer. While draining ACKs, a received `FLAG_NAK` is treated as an immediate, unconditional transfer failure – the receiving side has explicitly rejected a sequence number (see 3.2.3 for what triggers this), and the sender abandons the transfer rather than continuing to retransmit toward a peer that has already said no. A defensive check also treats a negative `recvfrom` return as a signal to stop polling for this cycle rather than looping indefinitely, specifically to avoid a Windows-specific failure mode where a UDP send to an unreachable port can later surface as a `WSAECONNRESET` on a subsequent receive call via a delayed ICMP Port Unreachable message.

`flush` calls `pollAcksAndRetransmit(true)` in a loop until the window is fully drained, giving the caller a genuine "every packet confirmed" completion signal.

`waitForClientReady` is the Passive-mode RETR handshake: the sender blocks waiting for a client's `FLAG_SYN` packet, captures the sender's address, and ACKs the SYN before normal sending can proceed.

RTT is sampled only from packets that were never retransmitted (Karn's Algorithm). On any timeout, the congestion window is halved and the retransmission timeout doubles, both bounded by the constants below; on a sustained run of clean ACKs equal to the current window size, the congestion window increases by 1.0. A set of read-only accessors (`getCongestionWindow`, `getTimeoutMs`, `getEstimatedRttMs`, `getDevRttMs`, `getCleanAckCount`) expose this internal state for direct inspection by the test suite, rather than requiring tests to infer congestion behavior indirectly from transfer timing alone.

Congestion Control & Timeout Parameters:

Field	Type	Purpose
RTT_ALPHA	0.125	EWMA smoothing factor for estimated RTT
RTT_BETA	0.25	EWMA smoothing factor for RTT deviation
MIN_TIMEOUT_MS	50	Clamping bounds on the computed retransmission timeout
MAX_TIMEOUT_MS	2000	
MIN_CWND	1.0	Congestion window floor and starting value
INITIAL_CWND	4.0	
MAX_RETRIES	10	Per-packet retry ceiling before the transfer is aborted with a hard failure

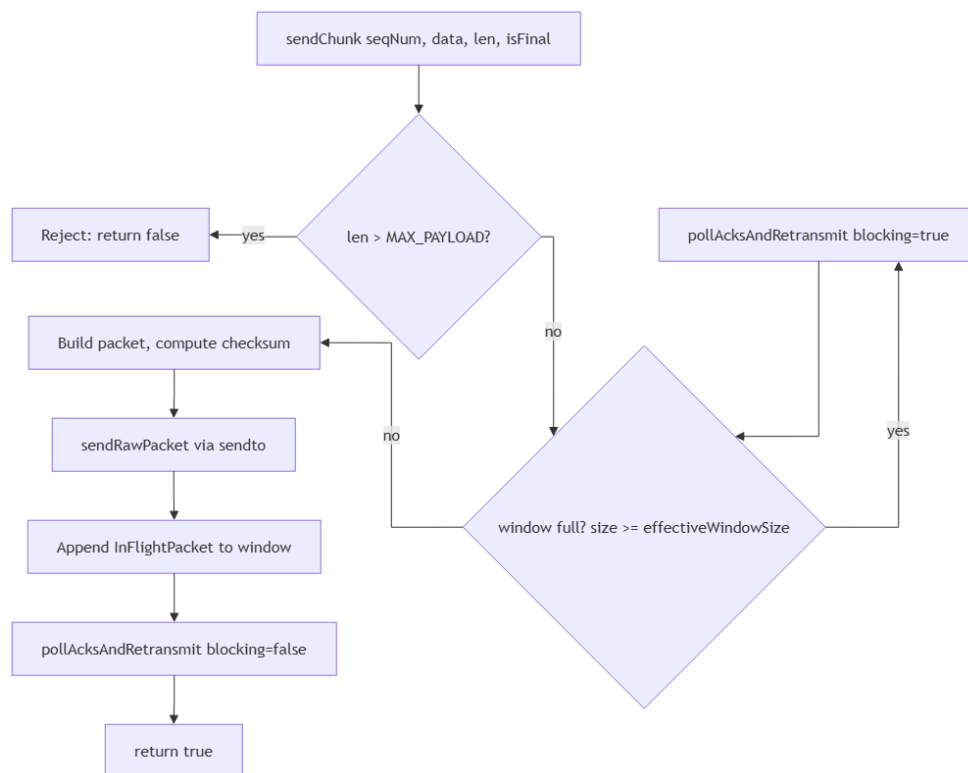


Figure 6: `sendChunk` High-Level Flow

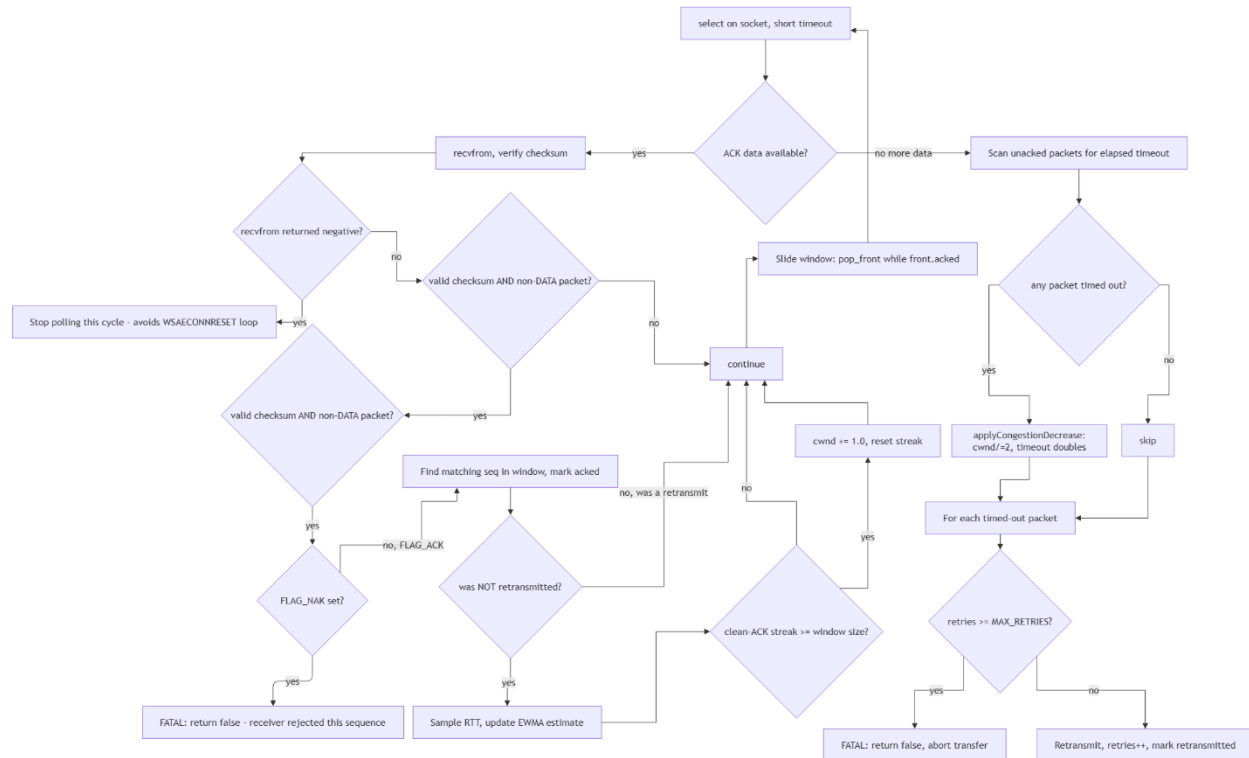


Figure 7: pollAcksAndRetransmit Subroutine

3.2.2. RdtReceiver – Selective Repeat Receive

RdtReceiver has two constructors – one that binds a fresh socket to a specific port (Active-mode upload), and one that reuses an already-bound socket (Passive-mode upload). All three upload commands (STOR, STOU, APPE) share this same construction logic via `startUploadTransfer` (Section 3.4), differing only in how the destination path is determined and what happens to the received data once the transfer completes.

initiateActiveHandshake is the Active-mode STOR counterpart to the sender's `waitForClientReady`, in the opposite direction: since the client already advertised its address via PORT, the server proactively sends a FLAG_SYN to that address and waits for a response, retrying up to HANDSHAKE_MAX_RETRIES times. If the response is itself a FLAG_DATA packet rather than a plain ACK, that datagram is buffered and is the first thing `receiveNext` checks for.

receiveNext first checks whether the next expected sequence number is already sitting in the out-of-order buffer, returning it immediately with no network wait. Otherwise it loops on `recvfrom`, silently discarding invalid-checksum packets, non-FLAG_DATA packets, or (once a peer address is locked in) packets from an unexpected address.

A packet whose sequence number is below `expected_seq` is immediately re-ACKed and ignored, regardless of where it would otherwise fall relative to the window. **Packets that fall**

outside the receive window are dropped silently, with no ACK sent at all – the window bound is an explicit constant, `RDT_RECEIVE_WINDOW` (10 – Section 2.2.4), and a packet is accepted only if `expected_seq ≤ seq_num < expected_seq + RDT_RECEIVE_WINDOW`. Ordering the checks this way (duplicate first, then window bound, then only ACK once a packet has cleared both) avoids ever sending an ACK for a packet that is about to be dropped.

The idle-timeout wait is abort-aware. When the caller has supplied an abort predicate (i.e. this receiver is running inside a transfer worker, per Figure 5), the socket's receive timeout is shortened to a 100 ms poll interval so the worker can notice `ABOR` promptly – but a plain socket-level timeout does not by itself mean "give up." `receiveNext` only actually returns `false` once the *real* idle budget (`DATA_IDLE_TIMEOUT_MS` normally, `CHAOS_DATA_IDLE_TIMEOUT_MS` under chaos-mode injection) has genuinely elapsed with no traffic; until then, each 100 ms poll just loops back around, checking the abort flag each time, rather than declaring a timeout.

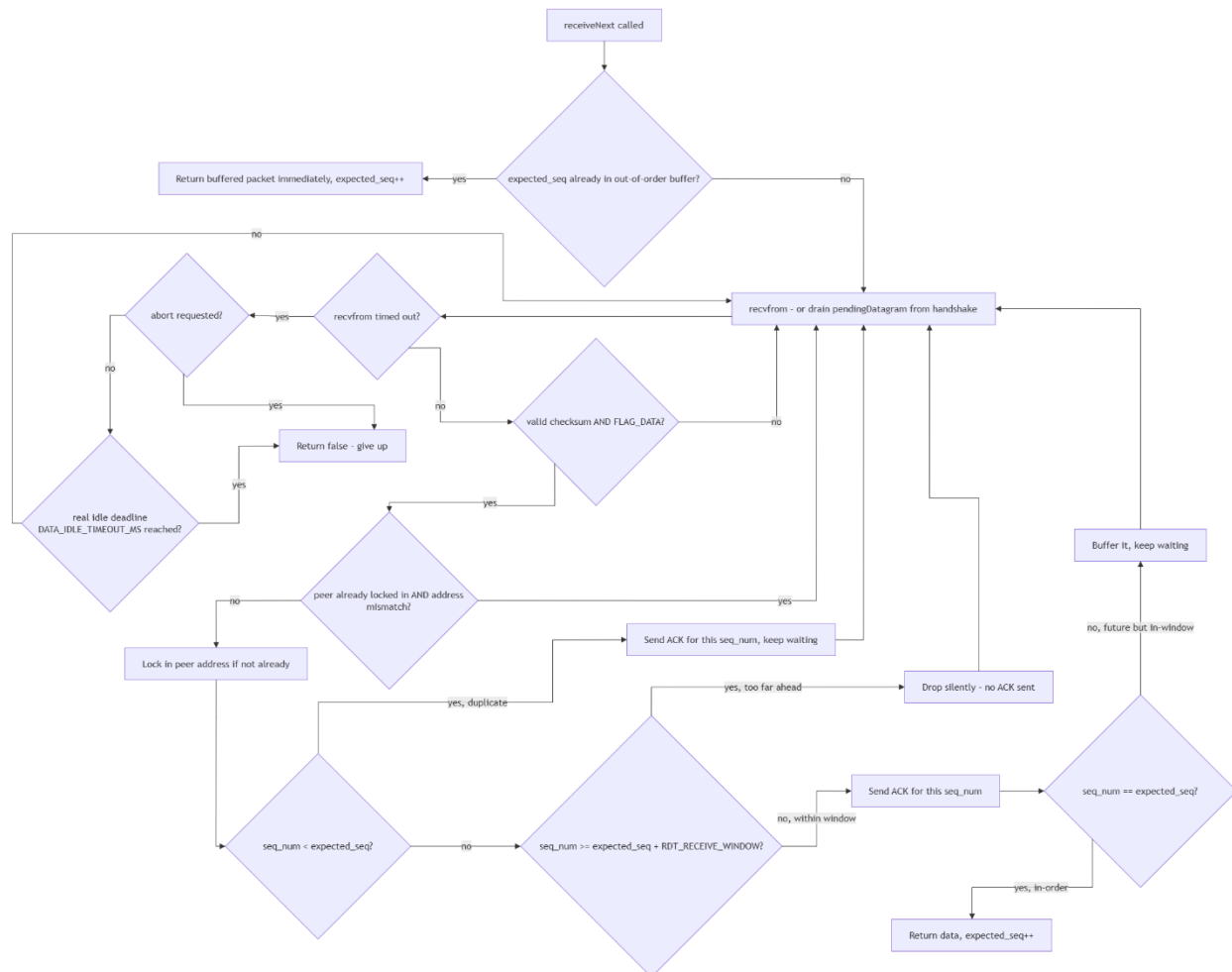


Figure 8: `receiveNext` Flow

3.2.3. Deferred Final-ACK on the Client's Download Path

This section describes a real mechanism found in `client/src/rdt/RdtReceiver.cpp` that has no counterpart on the server side.

In the design described in [3.2.2](#), a receiver ACKs a packet the instant it's validated – including the final chunk – before that data has necessarily been durably written to disk. For an upload (server receiving), that's an accepted, undocumented gap: the client could see “226 Transfer complete” before the server's `writer.commit()` (the atomic rename) has actually run. For a **download**, though, the same gap is directly observable and fixable by the client itself, since the client controls both ends of “did my own disk write succeed.” The client's `RdtReceiver` closes it.

`IRdtTransport::confirmReceive(seqNum, accepted)` (Section [2.2.5](#)) is overridden meaningfully only here. `receiveNext` still ACKs every non-final packet immediately, exactly as in [3.2.2](#). But when it receives the packet carrying `FLAG_FIN`, it does **not** ACK it yet – it sets `finalAcknowledgementPending = true`, records the sequence number, and returns the data to the caller (`DataChannelSession::receiveFile`, Section [3.4](#)) without sending anything on the wire for that packet. The caller is then responsible for calling `confirmReceive(seqNum, accepted)` once it knows the real outcome:

- If `ChunkedFileWriter::commit()` (the flush-close-then-atomic-rename sequence, Section 1.2 Phase 7 / 3.4) succeeds, the caller calls `confirmReceive(seqNum, true)`, which – because that sequence matches the pending one – finally sends the deferred `FLAG_ACK`.
- If the commit fails, or if any chunk's write to the temporary file fails (not just the final one), the caller calls `confirmReceive(seqNum, false)`, which sends a `FLAG_NAK` for that sequence instead. On the sender side, a received `FLAG_NAK` is treated as an unconditional fatal failure ([3.2.1](#)) – the sender abandons the transfer immediately rather than continuing to retry.

Note the asymmetry within the asymmetry: only the *final* packet's ACK is actually withheld pending commit; a write failure on an earlier chunk has already been ACKed by the time it's detected, so the NAK in that case isn't retracting an acknowledgment so much as it is a deliberate, explicit “stop, I can't continue” signal layered on top of the normal ACK stream. Both cases end the transfer the same way from the sender's perspective.

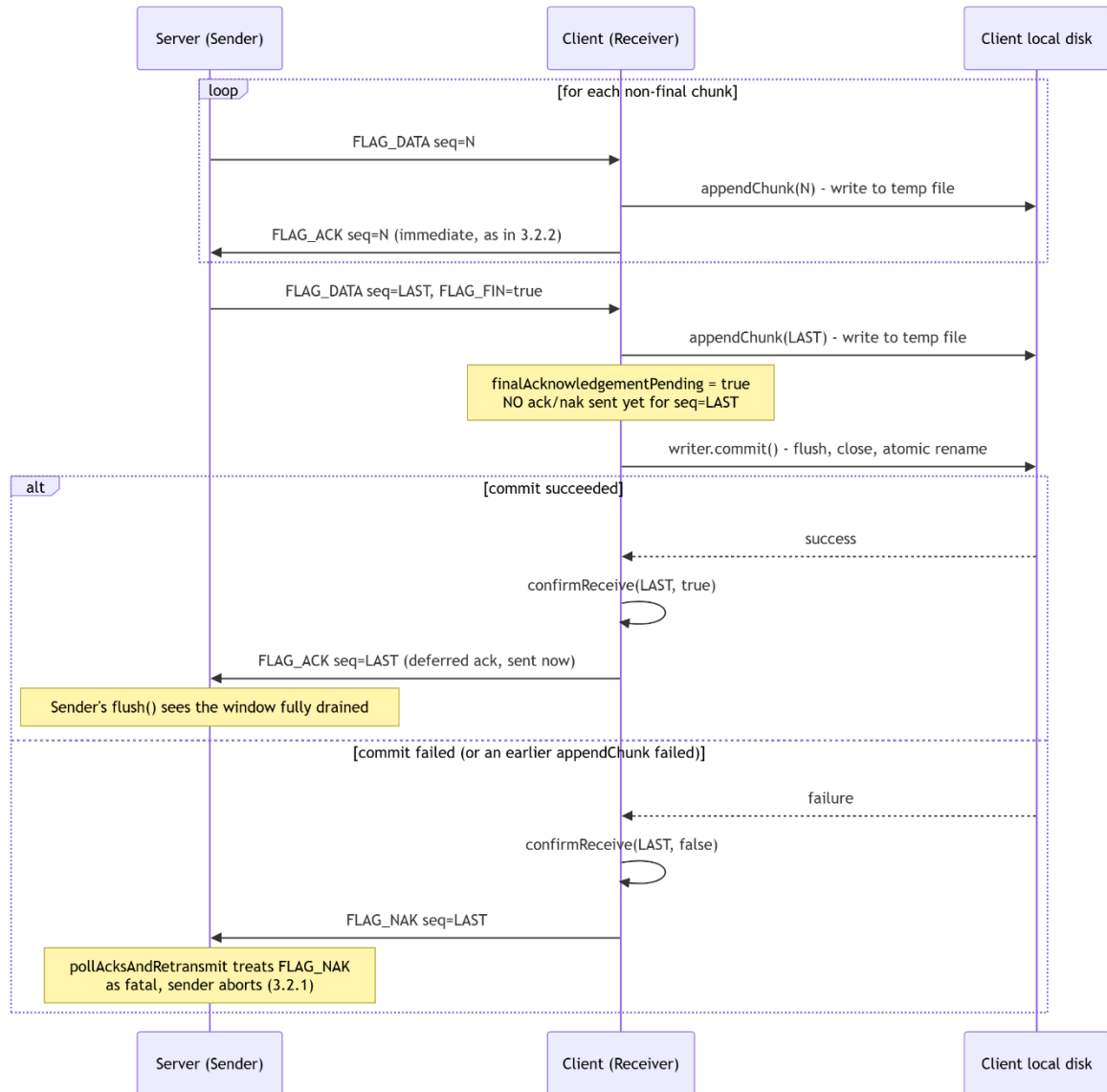


Figure 9: Deferred Final-ACK on Download (RETR, Client as Receiver)

3.3. Active/Passive Mode Toggle

This section documents the control-layer decision logic – specifically which code path `CommandDispatcher` takes based on the session's `dataChannelMode`.

PASV opens a server-side ephemeral UDP socket, storing it directly in the session, and sets `dataChannelMode = Passive`. PORT parses and validates the client's advertised address and additionally verifies – via `portMatchesControlPeer` – that the advertised IP matches the actual TCP control-connection's peer address, rejecting the command otherwise; this prevents a client from directing the server's data channel toward an unrelated third-party address (an FTP "bounce attack" vector).

When RETR, STOR, STOU or APPE is subsequently issued, the dispatcher first rejects the command if no mode was selected. Otherwise it branches: for Passive mode, it takes ownership of the already-open `pendingDataSocket` and constructs the corresponding transport with the existing-socket constructor; for Active mode, it constructs the fresh-socket variant using the client's validated address. The session's mode state resets to `None` immediately after this branch, so a mode selection is consumed exactly once per transfer. The whole process, rather than the transfer proceeding synchronously in place, it is handed off to a worker thread as shown in Figure 5 – the mode-toggle decision below determines *which transport* is built, not *how* the resulting transfer executes.

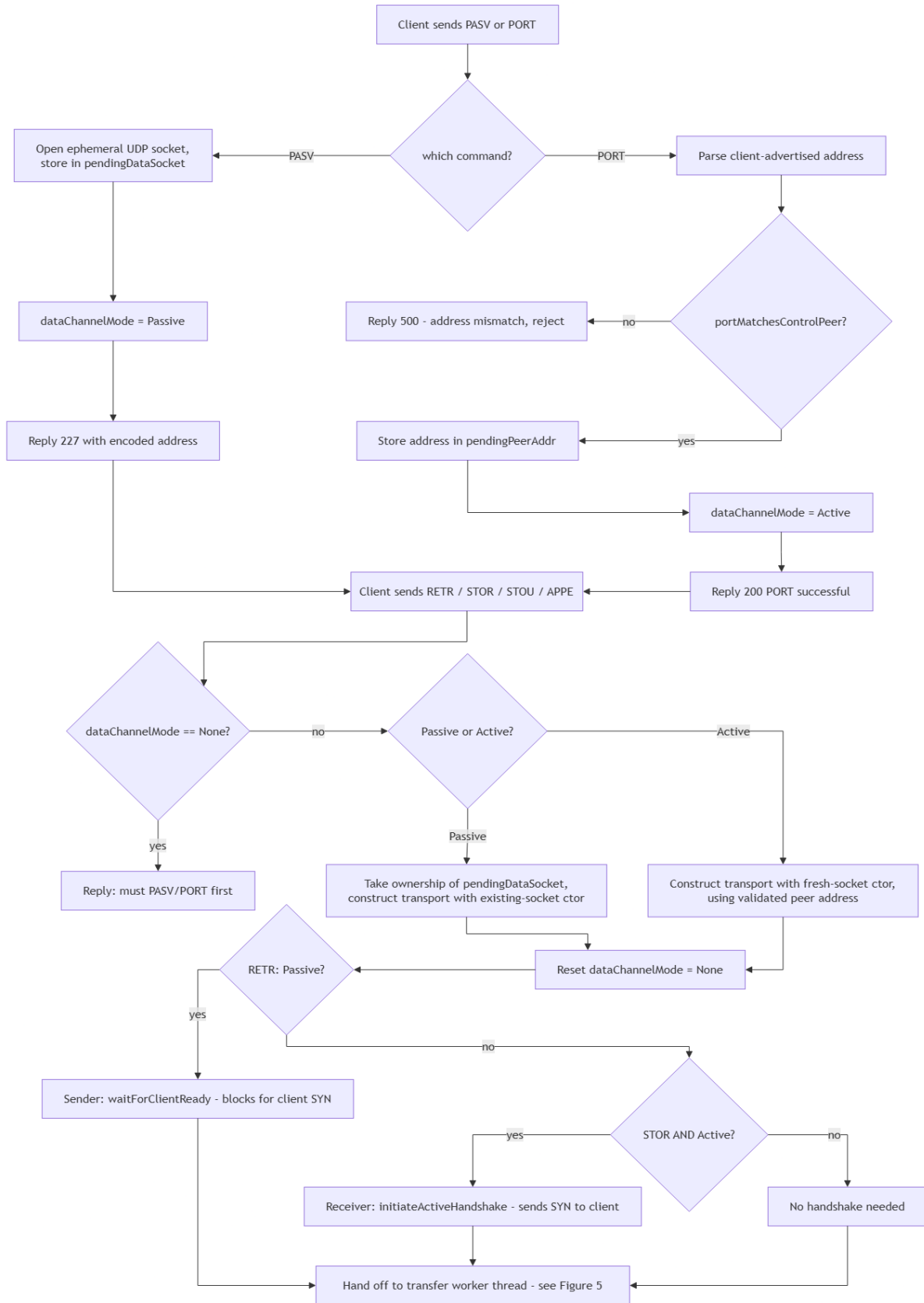


Figure 10: Active/Passive Mode Toggle

3.4. Unified Upload Handling: STOR / STOU / APPE

Once a transport is constructed (Figure 10) and handed to the transfer worker (Figure 5), the three upload commands converge on a single function, `startUploadTransfer`, parameterized by `UploadKind` (Section 2.3.3) rather than three separate implementations. The destination path – the one genuinely different input between the three – is determined before this shared logic runs: STOR resolves the client-supplied filename directly; STOU first calls the server-side unique-filename reservation (Figure 11); APPE resolves the client-supplied filename exactly as STOR does, but the kind passed through changes what happens once the data is fully received.

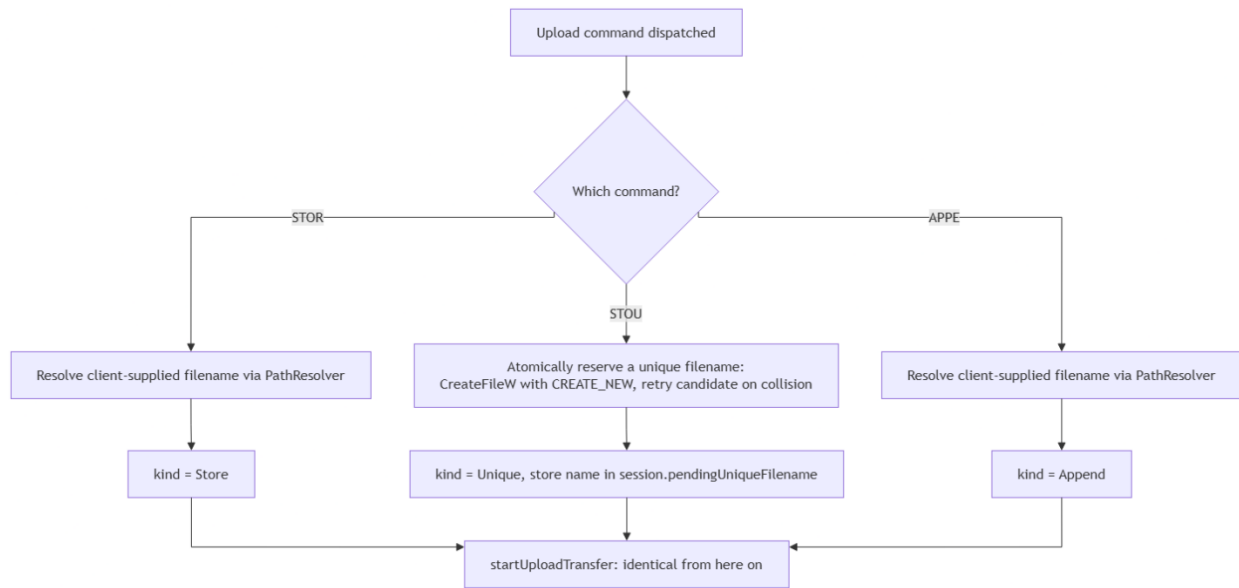


Figure 11: UploadKind Destination Resolution

STOU's reservation step is worth highlighting precisely because it avoids a check-then-create race entirely: `CreateFileW` with the `CREATE_NEW` flag atomically fails if a file with that exact name already exists, rather than separately checking existence and then creating – meaning two simultaneous STOU requests can never be issued the same filename, without needing any additional locking.

Once the transfer itself completes (the worker thread's `sendFile/receiveFile` call in Figure 5 returns), Store and Unique behave identically – the received data was streamed directly into the final destination's temporary file and is committed via the atomic rename already described for `ChunkedFileWriter`. Append takes one additional step after that same underlying receive completes, and always routes through a combined temp file, whether or not a destination already existed – there is no fast path that renames the staged upload straight to the destination, since `appendAtomically` needs a single atomic rename as its last step regardless, and building that file is simplest to implement as one unconditional copy loop rather than special-casing the empty-destination case.

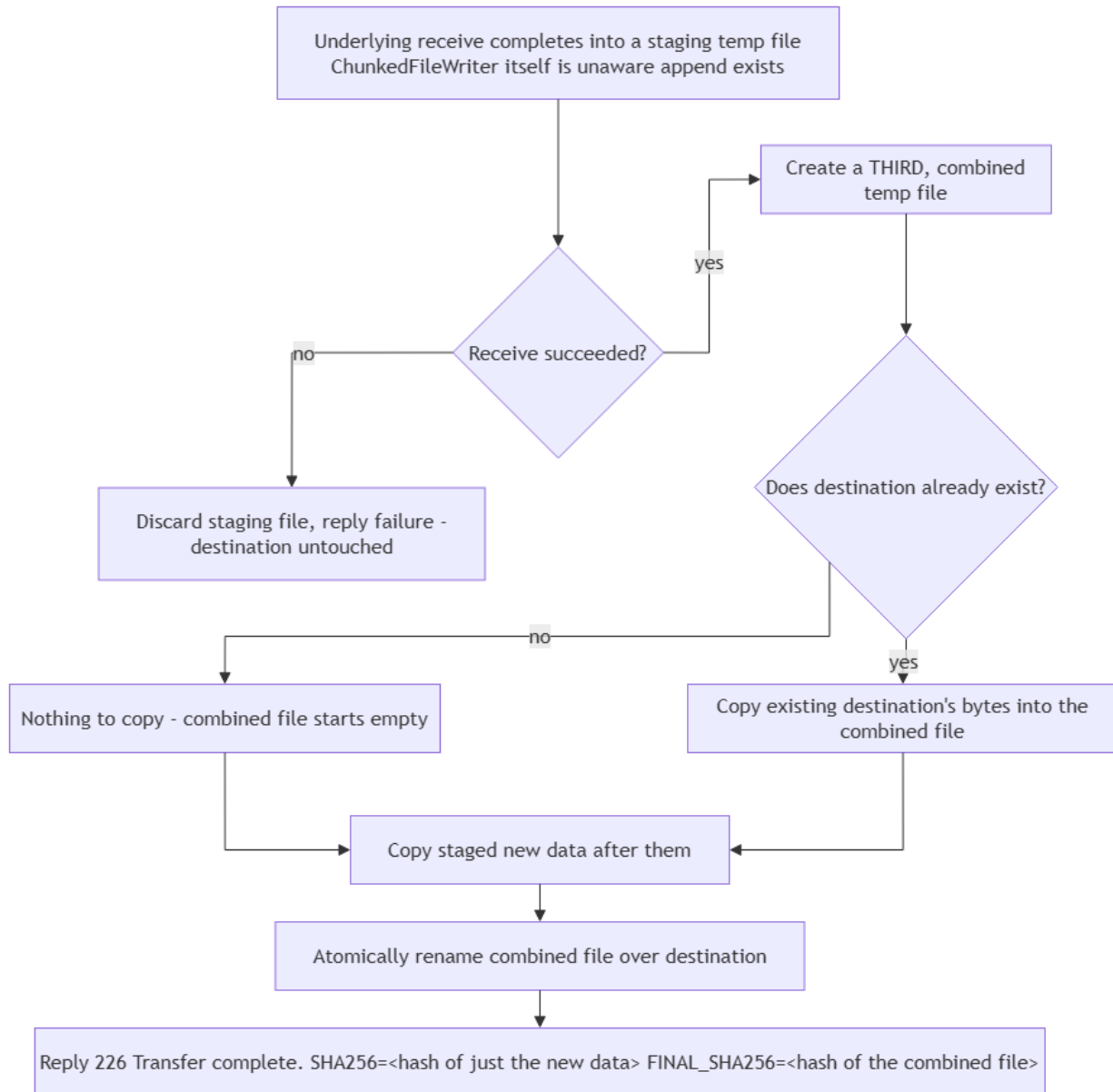


Figure 12: APPE Post-Receive Combination (*appendAtomically*)

This design deliberately keeps `ChunkedFileWriter` itself unmodified – it always just streams "the new data" into a temp file and atomically commits it, with no concept of "append" at all. The combination logic lives entirely in a separate step applied after a normal receive succeeds, at the cost of a second full-file copy for the existing content on a large append (once to seed the combined temp file, once already staged as new data) compared to a design that seeds the writer's temp file directly – a deliberate simplicity-over-I/O-efficiency tradeoff, not an oversight.

The completion reply's hash reporting differs meaningfully by command, via a shared `transferCompleteMessage(hash, uniqueFilename, finalHash)` helper. STOR/STOU in Binary mode report a single `SHA256=<...>` for the whole destination file (since there was nothing to append to); STOU additionally appends `Filename=<...>`, per the correction in Section 1.3. APPE in Binary mode reports **two** digests: `SHA256=<...>` is the hash of only the newly-received payload, and `FINAL_SHA256=<...>` is the hash of the resulting combined file – giving the client a way to verify both "did my upload arrive intact" and "does the server's file now match what I expect end-to-end" from a single reply line.

4. Task Assignment Matrix

4.1. Ownership Overview

The table below maps every major engineering component to its primary owner and supporting contributors, based on the team's original planning document and the project's commit history. Each member owned a distinct architectural layer for the full duration of the project; where teammates contributed to another member's layer, this is noted as supporting collaboration rather than a change of ownership, consistent with how the team worked throughout – components were frequently refined jointly once a first working version existed.

Component	Primary Owner	Supporting Contributions	Description
TCP control channel & accept loop	Lâm Hồng Hải Hoàng Lê	–	Server socket setup, listener, per-client thread spawning (<code>Session.cpp</code> , <code>server</code>)
Command parsing	Lâm Hồng Hải Hoàng Lê	–	Tokenizing raw command lines into verb + arguments
Command dispatch	Lâm Hồng Hải Hoàng Lê	Mai Phúc Hưng provided support with command-handler wiring and protocol-interface alignment for data-transfer commands	Routing parsed commands to their handlers, reply formatting
Asynchronous transfer execution	Mai Phúc Hưng	Lâm Hồng Hải Hoàng Lê provided support with control-loop integration, since this changes how the command dispatcher hands off long-running commands	Decouples RETR/STOR/STOU/APPE from the control-command loop so ABOR can interrupt an in-progress transfer

Unique & append upload variants	Mai Phúc Hưng	Đỗ Minh Khang provided support with the underlying ChunkedFileWriter/atomic-commit design these commands build on	Server-reserved unique filenames (STOU); non-destructive, atomically-committed appending to existing files (APPE)
Authentication & session state	Lâm Hồng Hải Hoàng Lê	Đỗ Minh Khang provided support with session-state field design during data-channel integration	Credential verification, per-client session fields, ClientRegistry
Client console entry point	Lâm Hồng Hải Hoàng Lê	Mai Phúc Hưng provided support with command-loop implementation and data-channel command routing	Interactive CLI, connects to server, drives the command loop
Path resolution & traversal safety	Đỗ Minh Khanh	–	Sandboxed path resolution, boundary enforcement
Directory operations	Đỗ Minh Khanh	–	PWD/CWD/MKD/RMD/LIST/STAT/SIZE/MDTM/DELE/RNFR/RNTO logic
Chunked file I/O	Đỗ Minh Khanh	–	Streaming binary reads/writes, out-of-order-safe reassembly
ASCII transfer mode	Đỗ Minh Khanh	–	TYPE A/TYPE I line-ending normalization
Integrity verification	Đỗ Minh Khanh	–	SHA-256 hashing via Windows CNG, end-to-end transfer verification
Data-channel orchestration	Đỗ Minh Khanh (server-side; initial client design)	Mai Phúc Hưng provided support with the client-side implementation, incorporating handshake and edge-case handling alongside protocol updates	Bridges filesystem I/O and the RDT transport interface
PASV/PORT socket setup	Đỗ Minh Khanh	Lâm Hồng Hải Hoàng Lê provided support with control-layer integration (mode-toggle dispatch, portMatchesControlPeer validation)	Ephemeral port binding, address parsing/formatting for both modes

RDT wire format	Mai Phúc Hưng	Đỗ Minh Khang provided support with shared-constant consolidation during a cross-module integration review	Packet header layout, checksum algorithm, shared protocol constants
Reliable transport	Mai Phúc Hưng	–	Sliding window, RTT estimation, AIMD congestion control, handshake logic for both transfer directions
Chaos/fault-injection testing harness	Mai Phúc Hưng	Đỗ Minh Khang provided support with end-to-end integrity verification tests exercising the harness	Configurable packet loss/corruption/latency injection for reliability testing
Build system (CMake, library structure)	All members	–	Client/server library separation, test target registration
Testing infrastructure (unit & integration tests)	All members	–	tests/ directory, per-module and full-pipeline test coverage
Project planning & documentation	Đỗ Minh Khanh	–	Initial team planning document, per-member implementation guides

4.2. Layer Summary

Lâm Hồng Hải Hoàng Lê – Control & Session Layer. Owns the TCP control channel end-to-end: connection acceptance, command parsing, command dispatch, authentication, and the client's interactive console. This layer is the integration point every other component ultimately connects through, and its command-dispatch table now covers the full approved command set, including asynchronous handling for long-running transfer commands.

Mai Phúc Hưng – Reliable Data Channel Layer. Owns the custom UDP reliability protocol in full: wire format, checksum verification, Selective Repeat windowing, RTT-based dynamic timeout estimation, and AIMD congestion control, along with the fault-injection harness used to validate it under adverse network conditions. Also extended the control layer with the asynchronous transfer-worker model underpinning ABOR, and implemented STOU, APPE, MODE, and HELP on top of the existing filesystem and data-channel infrastructure.

Đỗ Minh Khang – Filesystem, Crypto & Data-Channel Integration Layer. Owns the server's sandboxed filesystem access, all directory and file-metadata commands, binary and ASCII-mode chunked transfer, SHA-256 integrity verification, and the orchestration layer bridging file I/O to the reliable transport – along with the project's build system and initial planning documentation.

Across all three layers, components were refined collaboratively once a first working version existed – each member's supporting contributions to another's layer reflect the team's iterative, cross-reviewed development process rather than a reassignment of ownership.

5. Self-Assessment & Peer Evaluation

5.1. Lâm Hồng Hải Hoàng Lê

5.1.1. Self-Assessment

For this project, my primary responsibility was designing and implementing the server-client control channel. Although I initially faced a steep learning curve regarding the knowledge of the TCP protocol, I believe I have done sufficient work to deliver functional core components and contribute meaningfully to the project. After the completion of the program by everyone, I conducted thorough system testing to identify remaining issues and report back to my teammates.

During the process, I have recognized some of my limitations in terms of the speed I was progressing in and my initial efficiency in software development tools. Even though this slightly hindered our group's progress, the experience highlighted the importance of workflow optimization, and I have learned to utilize more of the available tools and resources to improve our collaboration during the last few days of the project.

5.1.2. Peer Evaluation

As our leader, Đỗ Minh Khang demonstrated exceptional project leadership by establishing a clear and well-defined structure to the project and assigning each task to a suitable teammate. His strategic skills were instrumental in allowing the team to finish the project within a tight schedule.

Mai Phúc Hưng provided invaluable collaborative support during the integration phase, working closely with Khang and myself to seamlessly interface his modules with the rest of the application. His ability to utilize his development tools to the utmost was reflected in the reliable, high-quality code he produced for the final system release.

5.2. Mai Phúc Hưng

5.2.1. Self-Assessment

During this project, I was mainly responsible for implementing and testing the RDT-related components. I worked on the required functions, checked the correctness of packet/header processing, and fixed issues that appeared during testing.

One important part of my work was debugging the RDT tests. For example, I encountered an issue related to the verification checksum of a serialized header. I reviewed the implementation, corrected the problem, and reran the test until the RdtTest passed successfully. This process helped me better understand how reliable data transfer works and how important careful serialization, checksum handling, and testing are in network programming.

I also worked to make sure that my implementation could integrate correctly with the other parts of the client-server system. Throughout the project, I improved my C++ programming, debugging, testing, and problem-solving skills. I also became more familiar with working in a team where different members are responsible for different components of the same system.

Overall, I believe I completed my assigned tasks successfully and made a meaningful contribution to the project. However, I could still improve by documenting my code more clearly, planning the implementation before coding, and communicating progress and technical issues with the team more frequently.

5.2.2. Peer Evaluation

My team members were responsible for different components of the client-server system, and each member contributed to the overall development of the project. We divided the work so that different parts could be developed in parallel while still being integrated into one complete system.

The team was supportive when technical problems occurred, especially when one component depended on another member's implementation. We communicated about requirements, interfaces, testing, and integration to reduce conflicts between different parts of the project.

Overall, I think the team worked effectively and each member contributed to the project. The cooperation between members helped us solve implementation and integration problems more efficiently. For future projects, we could improve by having more frequent progress updates, clearer documentation of interfaces and requirements, and earlier integration testing between components.

5.3. Đỗ Minh Khang

5.3.1. Self-Assessment

I believe I fulfilled my assigned responsibilities effectively throughout the project. I consistently met project deadlines, and the quality of my deliverables remained satisfactory. In addition to my technical contributions, I played an active role in coordinating the team's workflow and managing project activities. Although we encountered several avoidable issues during the initial and final stages of the project, the team's overall workflow remained reasonably efficient.

Despite these contributions, I recognize that there is still room for improvement in my technical abilities. At times, my programming skills limited the speed and effectiveness of my contributions, which may have affected the team's overall progress. Furthermore, I believe our final product could have achieved a higher standard if the team had benefited from stronger project leadership. As leadership is an area in which I still need to develop, some coordination challenges and workflow inefficiencies arose during the project.

5.3.2. Peer Evaluation

Throughout the project, Lâm Hồng Hải Hoàng Lê demonstrated a genuine commitment to understanding his assigned responsibilities and the technical aspects of his modules. Although he occasionally progressed more slowly than planned, he maintained a solid understanding of his tasks, and the quality of his code was consistently high.

One area for improvement is time and progress management, together with more proactive communication with the rest of the team. Because his development pace was sometimes slower than the project's schedule, other team members occasionally assisted with completing and integrating his work to ensure that project milestones were met. Strengthening these aspects would further enhance his overall effectiveness in future collaborative projects.

Mai Phúc Hưng demonstrated strong technical competence and consistently produced high-quality work throughout the project. He effectively utilized available tools and resources to develop well-polished solutions and was always willing to assist teammates by explaining technical concepts and providing support when needed. His collaborative attitude contributed positively to the team's development process.

However, there are areas where his workflow could be improved. He frequently accumulated substantial changes and submitted them shortly before deadlines instead of committing smaller, incremental updates throughout the development process. This practice made it more difficult to review and integrate his work. In addition, his enthusiasm sometimes led him to work beyond his assigned responsibilities by implementing features or modifications in other areas of the project. While these contributions were made with good intentions, they occasionally created uncertainty regarding task ownership and project coordination. Maintaining clearer boundaries while continuing to support the team would further improve collaboration in future projects.

5.4. Contribute Distribution

After several discussions and gathered all opinion from each member in the team, we agreed to distribute our contribution as follow:

ID	Name	Contribution (%)
25127052	Mai Phúc Hưng	35
25127207	Lâm Hồng Hải Hoàng Lê	32
25127363	Đỗ Minh Khang	33
Total		100

6. GenAI Usage & Code Refinement Log

Mai Phúc Hưng's log:

https://github.com/KangMing14/SocketProgramming/blob/main/docs/AI_Log/25127052.md

Lâm Hồng Hải Hoàng Lê's log:

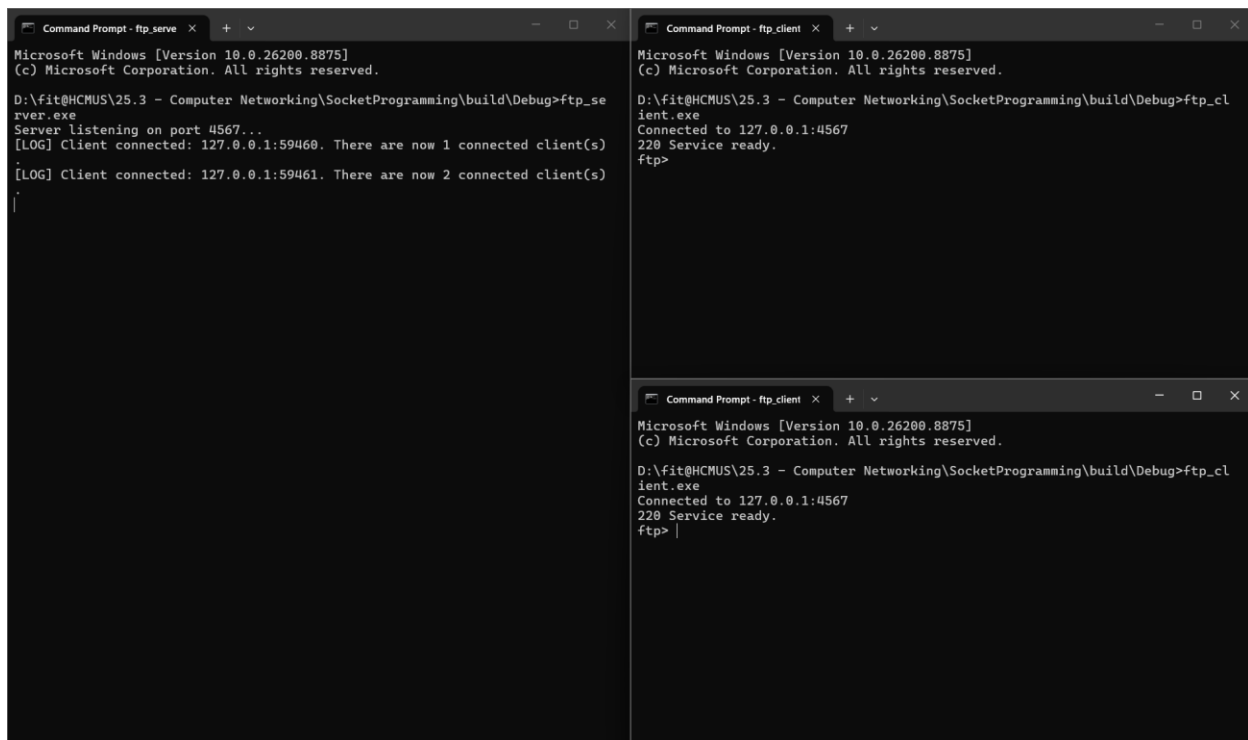
https://github.com/KangMing14/SocketProgramming/blob/main/docs/AI_Log/HoangLe.md

Đỗ Minh Khang's log:

https://github.com/KangMing14/SocketProgramming/blob/main/docs/AI_Log/MinhKhang.md

All the logs also stored within the submission zip file.

7. Application Demo Evidence



The image displays three separate Windows Command Prompt windows. The top-left window, titled 'Command Prompt - ftp_server', shows the execution of 'ftp_server.exe' and logs two client connections from 127.0.0.1. The top-right window, titled 'Command Prompt - ftp_client', shows the execution of 'ftp_client.exe', successful connection to 127.0.0.1:4567, and the prompt 'ftp>'. The bottom window, also titled 'Command Prompt - ftp_client', shows the same execution steps for a second client.

```
Command Prompt - ftp_server
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming\build\Debug>ftp_server.exe
Server listening on port 4567...
[LOG] Client connected: 127.0.0.1:59460. There are now 1 connected client(s)
.
[LOG] Client connected: 127.0.0.1:59461. There are now 2 connected client(s)
|

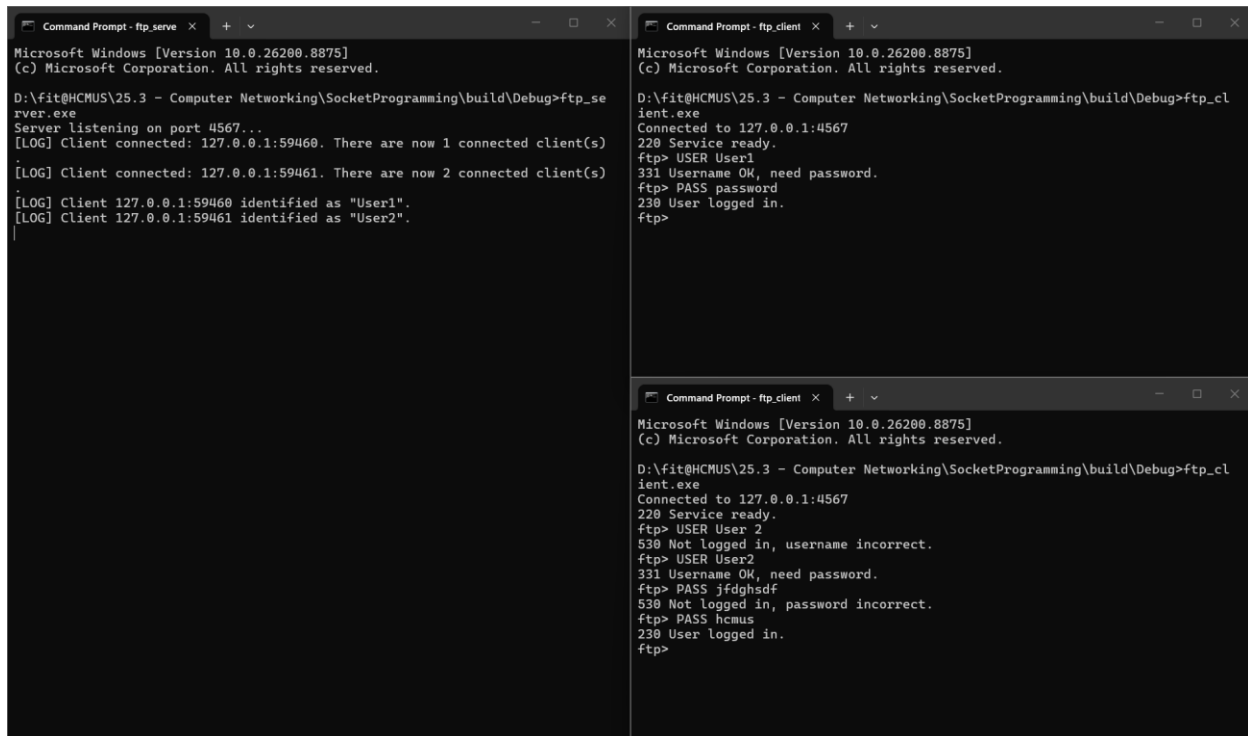
Command Prompt - ftp_client
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming\build\Debug>ftp_client.exe
Connected to 127.0.0.1:4567
220 Service ready.
ftp>

Command Prompt - ftp_client
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming\build\Debug>ftp_client.exe
Connected to 127.0.0.1:4567
220 Service ready.
ftp>
```

Figure A: The server and client programs after being started up. Two clients are connected to the server.



The figure consists of three screenshots of Windows Command Prompt windows. The top-left window, titled 'Command Prompt - ftp_server', shows the server listening on port 4567 and logging two clients: 127.0.0.1:59460 (User1) and 127.0.0.1:59461 (User2). The top-right window, titled 'Command Prompt - ftp_client', shows a client connecting to 127.0.0.1:4567, receiving a service ready message, and successfully logging in as 'User1' with password 'password'. The bottom window, also titled 'Command Prompt - ftp_client', shows a client attempting to log in as 'User 2' with an incorrect password 'jfdghsdf', receiving an error, and then successfully logging in as 'hcmus' with password 'hcmus'.

```
Microsoft Windows [Version 10.0.26280.8875]
(c) Microsoft Corporation. All rights reserved.

D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming\build\Debug>ftp_server.exe
Server listening on port 4567...
[LOG] Client connected: 127.0.0.1:59460. There are now 1 connected client(s)
[LOG] Client connected: 127.0.0.1:59461. There are now 2 connected client(s)
[LOG] Client 127.0.0.1:59460 identified as "User1".
[LOG] Client 127.0.0.1:59461 identified as "User2".

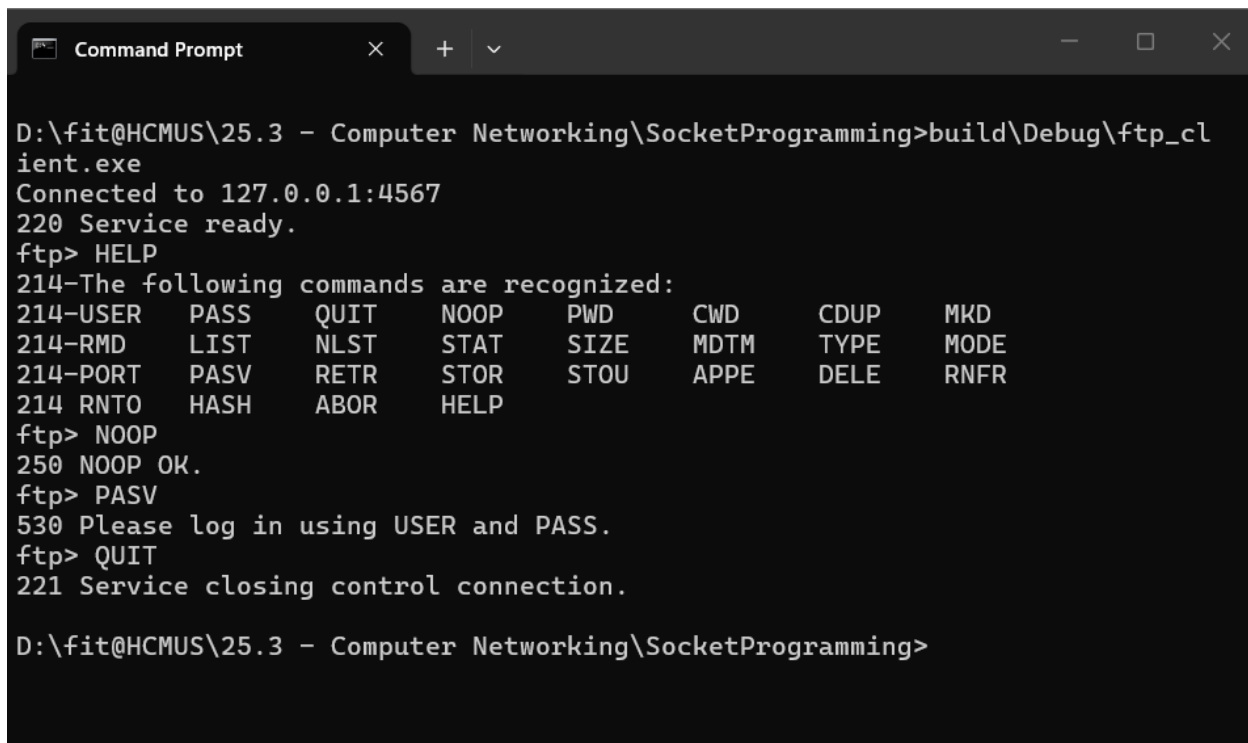
Microsoft Windows [Version 10.0.26280.8875]
(c) Microsoft Corporation. All rights reserved.

D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming\build\Debug>ftp_client.exe
Connected to 127.0.0.1:4567
220 Service ready.
ftp> USER User1
331 Username OK, need password.
ftp> PASS password
230 User logged in.
ftp>

Microsoft Windows [Version 10.0.26280.8875]
(c) Microsoft Corporation. All rights reserved.

D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming\build\Debug>ftp_client.exe
Connected to 127.0.0.1:4567
220 Service ready.
ftp> USER User 2
530 Not logged in, username incorrect.
ftp> USER User2
331 Username OK, need password.
ftp> PASS jfdghsdf
530 Not logged in, password incorrect.
ftp> PASS hcmus
230 User logged in.
ftp>
```

Figure B: Clients logging in via the USER and PASS commands.



The screenshot shows a Command Prompt window titled 'Command Prompt' with the path 'D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming>build\Debug\ftp_client.exe'. The client connects to 127.0.0.1:4567 and receives a service ready message. It then executes the HELP command, which lists recognized commands: USER, PASS, QUIT, NOOP, PWD, CWD, CDUP, MKD, RMD, LIST, NLST, STAT, SIZE, MDTM, TYPE, MODE, PORT, PASV, RETR, STOR, STOU, APPE, DELE, and RNFR. The client then executes NOOP, PASV, and QUIT, receiving appropriate responses from the server.

```
D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming>build\Debug\ftp_client.exe
Connected to 127.0.0.1:4567
220 Service ready.
ftp> HELP
214-The following commands are recognized:
214-USER    PASS    QUIT    NOOP    PWD    CWD    CDUP    MKD
214-RMD    LIST    NLST    STAT    SIZE    MDTM    TYPE    MODE
214-PORT    PASV    RETR    STOR    STOU    APPE    DELE    RNFR
214 RNT0    HASH    ABOR    HELP
ftp> NOOP
250 NOOP OK.
ftp> PASV
530 Please log in using USER and PASS.
ftp> QUIT
221 Service closing control connection.

D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming>
```

Figure C: Client can execute HELP, NOOP and QUIT commands without being logged in. Other commands, such as PASV, cannot be executed without logging in using USER and PASS.

```
ftp> PWD
257 "/" is the current directory.
ftp> CWD temp
250 Directory changed to temp
ftp> PWD
257 "/temp" is the current directory.
ftp> CDUP
250 Directory changed to server_root
ftp> LIST
250--rwxrwxrwx 26 active-upload.txt
250--rwxrwxrwx 10485760 large-upload.bin
250--rwxrwxrwx 10485760 remote-large.bin
250--rwxrwxrwx 26 remote.txt
250-drwxrwxrwx 0 temp
250--rwxrwxrwx 26 text-copy.txt
250 -rwxrwxrwx 26 upload_0001.bin
ftp> MKD new
257 "new" created
ftp> SIZE remote.txt
213 26
ftp> HASH text-copy.txt
213 SHA-256 4eb7400ffd091618aea799980a91f71ae8a7559b55cbb3d06b4716ece1fca229
ftp> STAT
211 Server status: OK.1 client(s) connected.
ftp> STAT active-upload.txt
213 active-upload.txt: file, 26 bytes
ftp> |
```

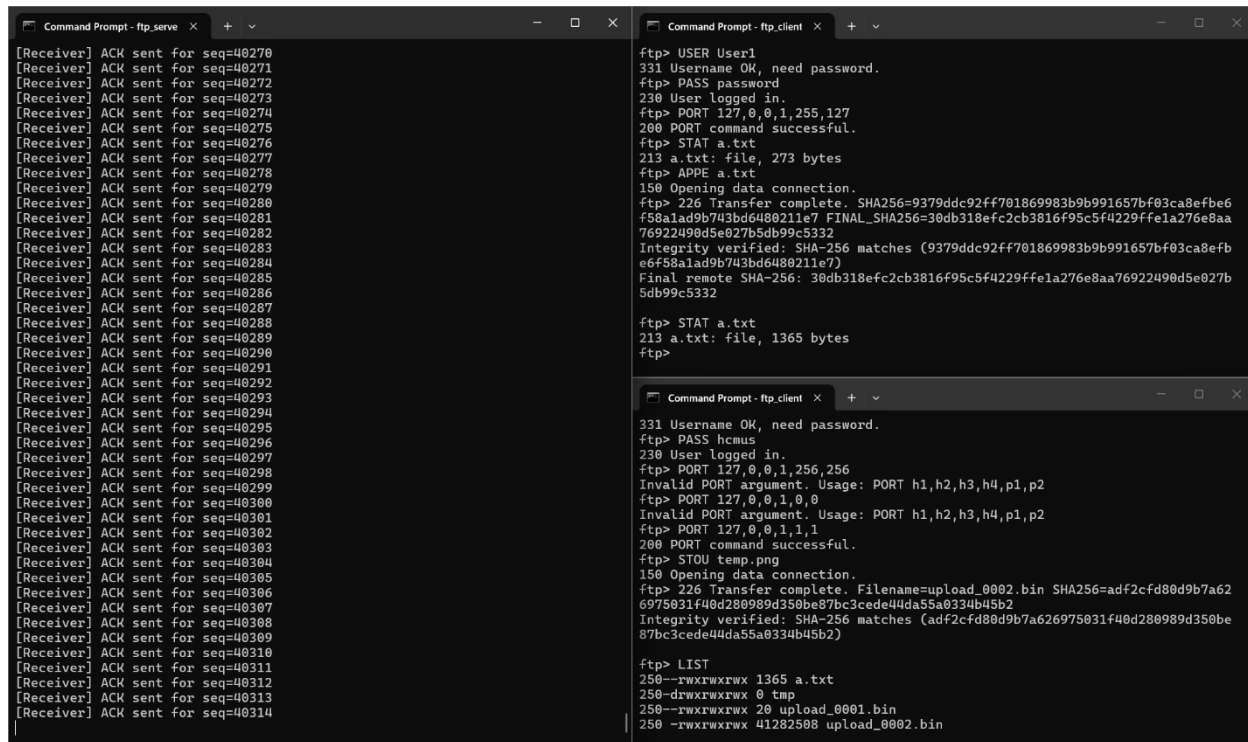
Figure D: Client can execute control-only commands, such as PWD, CWD, CDUP, LIST, MKD, SIZE, STAT.

```
Command Prompt - build\Dei x + v
[Receiver] ACK sent for seq=10195
[Receiver] ACK sent for seq=10196
[Receiver] ACK sent for seq=10197
[Receiver] ACK sent for seq=10198
[Receiver] ACK sent for seq=10199
[Receiver] ACK sent for seq=10200
[Receiver] ACK sent for seq=10201
[Receiver] ACK sent for seq=10202
[Receiver] ACK sent for seq=10203
[Receiver] ACK sent for seq=10204
[Receiver] ACK sent for seq=10205
[Receiver] ACK sent for seq=10206
[Receiver] ACK sent for seq=10207
[Receiver] ACK sent for seq=10208
[Receiver] ACK sent for seq=10209
[Receiver] ACK sent for seq=10210
[Receiver] ACK sent for seq=10211
[Receiver] ACK sent for seq=10212
[Receiver] ACK sent for seq=10213
[Receiver] ACK sent for seq=10214
[Receiver] ACK sent for seq=10215
[Receiver] ACK sent for seq=10216
[Receiver] ACK sent for seq=10217
[Receiver] ACK sent for seq=10218
[Receiver] ACK sent for seq=10219
[Receiver] ACK sent for seq=10220
[Receiver] ACK sent for seq=10221
[Receiver] ACK sent for seq=10222
[Receiver] ACK sent for seq=10223
[Receiver] ACK sent for seq=10224
[Receiver] ACK sent for seq=10225
[Receiver] ACK sent for seq=10226
[Receiver] ACK sent for seq=10227
[Receiver] ACK sent for seq=10228
[Receiver] ACK sent for seq=10229
[Receiver] ACK sent for seq=10230
[Receiver] ACK sent for seq=10231
[Receiver] ACK sent for seq=10232
[Receiver] ACK sent for seq=10233
[Receiver] ACK sent for seq=10234
[Receiver] ACK sent for seq=10235
[Receiver] ACK sent for seq=10236
[Receiver] ACK sent for seq=10237
[Receiver] ACK sent for seq=10238
[Receiver] ACK sent for seq=10239

Command Prompt - build\Dei x + v
D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming>build\Debug\ftp_cl
ient.exe
Connected to 127.0.0.1:4567
220 Service ready.
ftp> USER User1
331 Username OK, need password.
ftp> PASS password
230 User logged in.
ftp> PASV
227 Entering Passive Mode (127,0,0,1,246,215).
ftp> STOR ./large-upload.bin
150 Opening data connection.
ftp> 226 Transfer complete. SHA256=e5b844cc57f57094ea4585e235f36c78c1cd2226
2bb89d53c94dc4d6b3e55d
Integrity verified: SHA-256 matches (e5b844cc57f57094ea4585e235f36c78c1cd22
262bb89d53c94dc4d6b3e55d)

Command Prompt - build\Dei x + v
D:\fit@HCMUS\25.3 - Computer Networking\SocketProgramming>build\Debug\ftp_cl
ient.exe
Connected to 127.0.0.1:4567
220 Service ready.
ftp> USER User2
331 Username OK, need password.
ftp> PASS hcmus
230 User logged in.
ftp> PASV
227 Entering Passive Mode (127,0,0,1,219,65).
ftp> RETR remote-large.bin
150 Opening data connection.
ftp> 226 Transfer complete. SHA256=e5b844cc57f57094ea4585e235f36c78c1cd2226
2bb89d53c94dc4d6b3e55d
Integrity verified: SHA-256 matches (e5b844cc57f57094ea4585e235f36c78c1cd22
262bb89d53c94dc4d6b3e55d)
```

Figure E: Clients enter Passive Mode using PASV. While a client sends a large file to the server, another client can retrieve a file from the server without affecting the other client.



```
Command Prompt - ftp_serve x + -
[Receiver] ACK sent for seq=40270
[Receiver] ACK sent for seq=40271
[Receiver] ACK sent for seq=40272
[Receiver] ACK sent for seq=40273
[Receiver] ACK sent for seq=40274
[Receiver] ACK sent for seq=40275
[Receiver] ACK sent for seq=40276
[Receiver] ACK sent for seq=40277
[Receiver] ACK sent for seq=40278
[Receiver] ACK sent for seq=40279
[Receiver] ACK sent for seq=40280
[Receiver] ACK sent for seq=40281
[Receiver] ACK sent for seq=40282
[Receiver] ACK sent for seq=40283
[Receiver] ACK sent for seq=40284
[Receiver] ACK sent for seq=40285
[Receiver] ACK sent for seq=40286
[Receiver] ACK sent for seq=40287
[Receiver] ACK sent for seq=40288
[Receiver] ACK sent for seq=40289
[Receiver] ACK sent for seq=40290
[Receiver] ACK sent for seq=40291
[Receiver] ACK sent for seq=40292
[Receiver] ACK sent for seq=40293
[Receiver] ACK sent for seq=40294
[Receiver] ACK sent for seq=40295
[Receiver] ACK sent for seq=40296
[Receiver] ACK sent for seq=40297
[Receiver] ACK sent for seq=40298
[Receiver] ACK sent for seq=40299
[Receiver] ACK sent for seq=40300
[Receiver] ACK sent for seq=40301
[Receiver] ACK sent for seq=40302
[Receiver] ACK sent for seq=40303
[Receiver] ACK sent for seq=40304
[Receiver] ACK sent for seq=40305
[Receiver] ACK sent for seq=40306
[Receiver] ACK sent for seq=40307
[Receiver] ACK sent for seq=40308
[Receiver] ACK sent for seq=40309
[Receiver] ACK sent for seq=40310
[Receiver] ACK sent for seq=40311
[Receiver] ACK sent for seq=40312
[Receiver] ACK sent for seq=40313
[Receiver] ACK sent for seq=40314

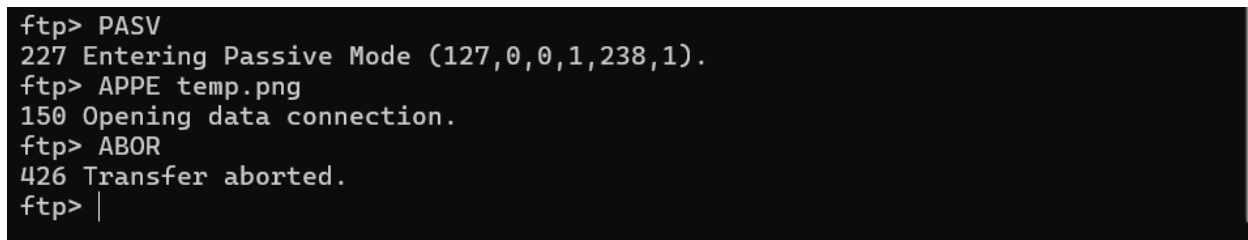
Command Prompt - ftp_client x + -
ftp> USER User1
331 Username OK, need password.
ftp> PASS password
230 User logged in.
ftp> PORT 127,0,0,1,255,127
200 PORT command successful.
ftp> STAT a.txt
213 a.txt: file, 273 bytes
ftp> APPE a.txt
150 Opening data connection.
ftp> 226 Transfer complete. SHA256=9379ddc92ff701869983b9b991657bf03ca8efbe6
f58a1ad9b743bd6480211e7 FINAL_SHA256=30db318efc2cb3816f95c5f4229ffela276e8aa
76922490d5e027b5db99c5332
Integrity verified: SHA-256 matches (9379ddc92ff701869983b9b991657bf03ca8efb
e6f58a1ad9b743bd6480211e7)
Final remote SHA-256: 30db318efc2cb3816f95c5f4229ffela276e8aa76922490d5e027b
5db99c5332

ftp> STAT a.txt
213 a.txt: file, 1365 bytes
ftp>

Command Prompt - ftp_client x + -
331 Username OK, need password.
ftp> PASS hcmus
230 User logged in.
ftp> PORT 127,0,0,1,256,256
Invalid PORT argument. Usage: PORT h1,h2,h3,h4,p1,p2
ftp> PORT 127,0,0,1,0,0
Invalid PORT argument. Usage: PORT h1,h2,h3,h4,p1,p2
ftp> PORT 127,0,0,1,1,1
200 PORT command successful.
ftp> STOU temp.png
150 Opening data connection.
ftp> 226 Transfer complete. Filename=upload_0002.bin SHA256=adf2cfd80d9b7a62
6975031f40d280989d350be87bc3cede44da55a0334b45b2
Integrity verified: SHA-256 matches (adf2cfd80d9b7a626975031f40d280989d350be
87bc3cede44da55a0334b45b2)

ftp> LIST
250--rw-rw-rw- 1365 a.txt
250--d-rwxrwxrwx 0 tmp
250--rw-rw-rw- 20 upload_0001.bin
250 -rw-rw-rw- 41282508 upload_0002.bin
```

Figure F: Clients enter Active Mode using PORT. A client executes APPE which appends data to the existing file on the server, and the other client executes STOU which creates a new file with a unique filename.



```
ftp> PASV
227 Entering Passive Mode (127,0,0,1,238,1).
ftp> APPE temp.png
150 Opening data connection.
ftp> ABOR
426 Transfer aborted.
ftp> |
```

Figure G: Client can abort the data transferring process.

8. Tools used

8.1. Libraries

- Socket: Winsock2 (ws2_32.lib, winsock2.h)
- Hashing: MS Cryptography API: Next Generation (CNG/bcrypt)

* Both are native Windows OS components, not third-party libraries. While direct usage of these specific APIs ties the current source code to the Windows environment, the application's underlying logic can be made portable to other operating systems by introducing standard cross-platform abstractions (such as POSIX sockets for networking and OpenSSL for cryptography).

8.2. Applications/Websites

- Documenting: MS Word
- Collaboration: GitHub, Messenger
- Code Editor/IDE: Visual Studio Code, Visual Studio 2022
- GenAI: Claude, Antigravity, Codex